Document Information	
Filename	TP_F101U-00026_User_Guide_Persistence
Status	Release
Revision Date	July 17, 2023
Version	07.00
Document ID	TP_F101U-00026

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Table of contents

1 Introduction ... 6
1.1 Notational Conventions ... 8
2 Setup ... 9
2.1 Configuration ... 9
2.2 Data Types in XML Configuration Files ... 14
2.3 Build Process ... 16
2.3.1 HOWTO be a Client of Persistence Client Library ... 16
2.3.2 HOWTO be a Client of Persistence Admin Service ... 17
2.4 Tools/Equipment Needed ... 18
2.5 Setup Instructions ... 18
2.6 Setup Troubleshooting ... 18
3 Features/Components ... 20
3.1 File Data ... 20
3.1.1 Overview ... 20
3.1.1.1 Platform Content Overview ... 20
3.1.1.2 Product Expectations ... 20
3.1.1.3 Extending Platform Functionality ... 20
3.1.2 Setup ... 20
3.1.2.1 Tools/Equipment Needed ... 20
3.1.2.2 Setup Instructions ... 20
3.1.2.3 Setup Troubleshooting ... 20
3.1.3 Usage ... 20
3.1.3.1 Basic Instructions ... 20
3.1.3.2 Detailed Instructions ... 20
3.1.3.3 Testing/Error Reporting ... 22
3.2 Key-Value Pair Data ... 23
3.2.1 Overview ... 23
3.2.1.1 Platform Content Overview ... 23
3.2.1.2 Product Expectations ... 23
3.2.1.3 Extending Platform Functionality ... 23
3.2.2 Setup ... 23
3.2.2.1 Tools/Equipment Needed ... 23
3.2.2.2 Setup Instructions ... 23
3.2.2.3 Setup Troubleshooting ... 23
3.2.3 Usage ... 23
3.2.3.1 Basic Instructions ... 23
3.2.3.2 Detailed Instructions ... 23
3.2.3.3 Testing/Error Reporting ... 24
3.3 Data Access Permissions ... 25

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.1 Overview 25
3.3.1.1 Platform Content Overview 25
3.3.1.2 Product Expectations 25
3.3.1.3 Extending Platform Functionality 25
3.3.2 Setup 25
3.3.2.1 Tools/Equipment Needed 25
3.3.2.2 Setup Instructions 25
3.3.2.3 Setup Troubleshooting 26
3.3.3 Usage 26
3.3.3.1 Basic Instructions 26
3.3.3.2 Detailed Instructions 26
3.3.3.3 Testing/Error Reporting 28
3.4 Data Change Notifications 29
3.4.1 Overview 29
3.4.1.1 Platform Content Overview 29
3.4.1.2 Product Expectations 29
3.4.1.3 Extending Platform Functionality 29
3.4.2 Setup 29
3.4.2.1 Tools/Equipment Needed 30
3.4.2.2 Setup Instructions 30
3.4.2.3 Setup Troubleshooting 30
3.4.3 Usage 30
3.4.3.1 Basic Instructions 30
3.4.3.2 Detailed Instructions 30
3.4.3.3 Testing/Error Reporting 31
3.5 Data Corruption Detection 33
3.5.1 Overview 33
3.5.1.1 Platform Content Overview 33
3.5.1.2 Product Expectations 34
3.5.1.3 Extending Platform Functionality 34
3.5.2 Setup 34
3.5.2.1 Tools/Equipment Needed 34
3.5.2.2 Setup Instructions 34
3.5.2.3 Setup Troubleshooting 34
3.5.3 Usage 34
3.5.3.1 Basic Instructions 34
3.5.3.2 Detailed Instructions 34
3.5.3.3 Testing/Error Reporting 37
3.6 Persistence Data Product Overrides 38
3.6.1 Overview 38
3.6.1.1 Platform Content Overview 38
3.6.1.2 Product Expectations 38
3.6.1.3 Extending Platform Functionality 38

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.6.2 Setup 38
3.6.2.1 Tools/Equipment Needed 38
3.6.2.2 Setup Instructions 38
3.6.2.3 Setup Troubleshooting 38
3.6.3 Usage 38
3.6.3.1 Basic Instructions 38
3.6.3.2 Detailed Instructions 39
3.6.3.3 Testing/Error Reporting 41
3.7 Persistence Backup and Restore 42
3.7.1 Overview 42
3.7.1.1 Platform Content Overview 42
3.7.1.2 Product Expectations 42
3.7.1.3 Extending Platform Functionality 42
3.7.2 Setup 42
3.7.2.1 Tools/Equipment Needed 42
3.7.2.2 Setup Instructions 42
3.7.2.3 Setup Troubleshooting 42
3.7.3 Usage 42
3.7.3.1 Basic Instructions 42
3.7.3.2 Detailed Instructions 42
3.7.3.3 Testing/Error Reporting 45
3.8 Restoring Defaults 46
3.8.1 Overview 46
3.8.1.1 Platform Content Overview 46
3.8.1.2 Product Expectations 46
3.8.1.3 Extending Platform Functionality 46
3.8.2 Setup 46
3.8.2.1 Tools/Equipment Needed 46
3.8.2.2 Setup Instructions 46
3.8.2.3 Setup Troubleshooting 46
3.8.3 Usage 47
3.8.3.1 Basic Instructions 47
3.8.3.2 Detailed Instructions 47
3.8.3.3 Testing/Error Reporting 48
3.9 Persistence Data Field Update/Software Update 49
3.9.1 Overview 49
3.9.1.1 Platform Content Overview 49
3.9.1.2 Product Expectations 51
3.9.1.3 Extending Platform Functionality 51
3.9.2 Setup 51
3.9.2.1 Tools/Equipment Needed 51
3.9.2.2 Setup Instructions 51
3.9.2.3 Setup Troubleshooting 51

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.9.3 Usage 51
3.9.3.1 Basic Instructions 51
3.9.3.2 Detailed Instructions 51
3.9.3.3 Testing/Error Reporting 55
4 Known Issues 55
5 FAQs 56
5.1 Secure files 56
5.1.1 Opening a secure file for writing 56
6 Troubleshooting/Tips and Tricks 56
7 References 57
7.1 Abbreviations/Acronyms/Definitions 57
7.2 Platform Documentation 57
7.3 Website links 57
7.4 Appendix XML Configuration File Example 57
8 Revision History 61

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

1 Introduction

This document provides usage information for Persistence. It is intended for users of this service provided by the Telematics Platform. Persistence component is designed to store configured data over power cycles. Persistence allows the software components to define the access rights to the data and it supports registration for change notifications.

Persistence can be divided in 3 big components: **Persistence Client Library (PCL), Persistence Admin Service (PAS)** and **Persistence Installer**. The combination of PCL and PAS provide the feature explained in detail in this document.

- Persistence Client Library is the component that provides access to the persisted data after it is already installed.
- Persistence Admin Service is the component that provides admin rights to privileged users and facilitates the update in the field. Software Loading Component uses PAS to perform the Persistence update in the field during a software update.

In this document, persisted data is referred to as **resources**. Persistence supports and handles 2 types of resources, **files** and **key-value pairs**.

Persistence stores the data in a structured way in the Linux file system. The classification of the data is done based on the storage type: **public**, **group** or **local**.

The **public** storage type is designed for data that needs to be available to all users in the system. No access permissions restrictions are enforced for this storage type.

The **group** shared storage type is designed for data that needs to be shared by users within a specific Linux group. When a user tries to access group data, Persistence will check if the user is part of the group either as a primary or supplementary one.

The **local** storage type is designed for data visible only to the software component that defines it in the first place. A combination of application name, user and group is used to protect the access to this type of data.

Persistence Client Library

The following diagram depicts how PCL fits into the platform:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

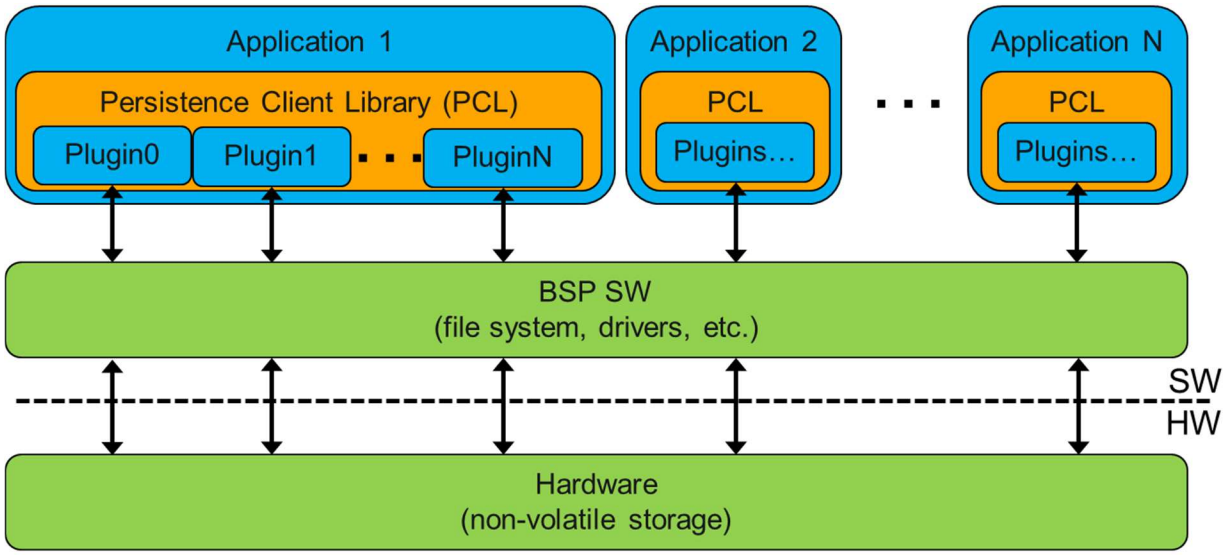


Figure 1 PCL Integration in the Platform

Persistence Admin Service

The following diagram depicts how PAS fits into the platform:

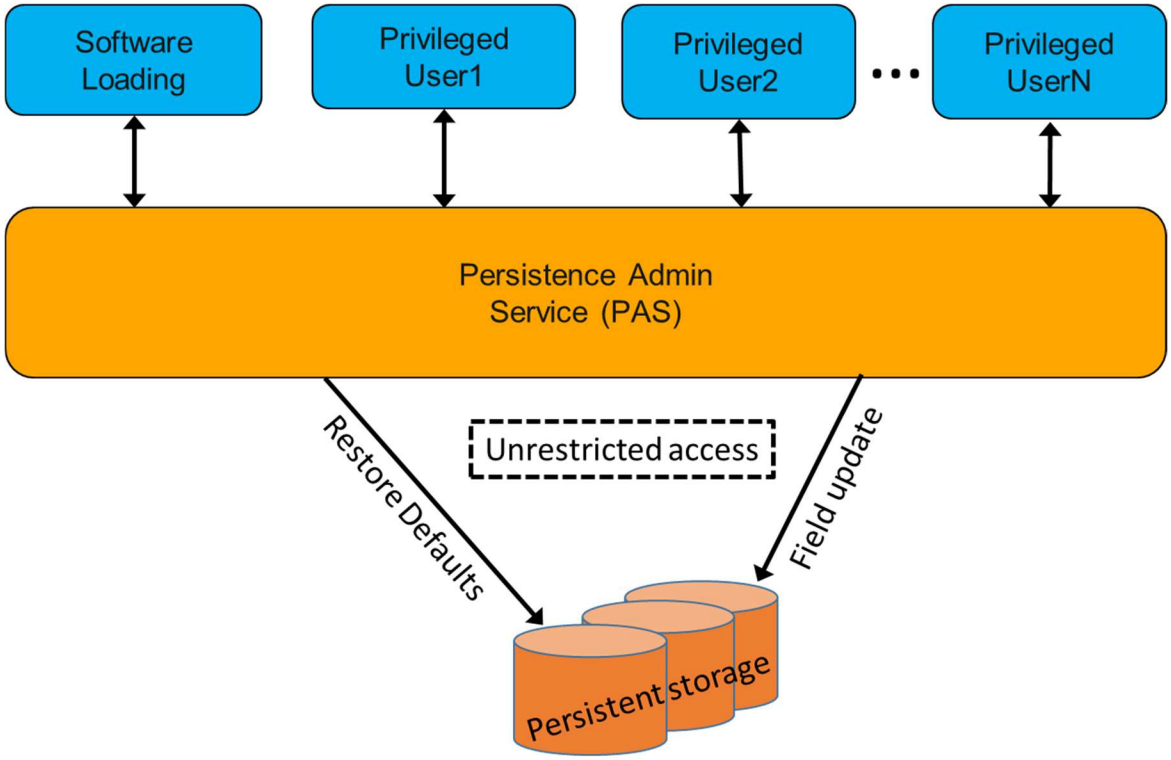


Figure 2 PAS Integration in the Platform

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	A link to an external reference (e.g., website, document)

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend	Telematics Platform Product Specific 3rd Party

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2 Setup

2.1 Configuration

For other software components to make use of Persistence, they need to design configuration files to define their own resources.

Every software component in the system can configure persistent data to be created on the target for all types of persistent data that may be required for run-time operation of the component. This configured persistent data can be complete files or key-value pair type of data. Any number of persistent file resources or persistent key-value pair resources can be configured in a single Persistence XML configuration file.

A persistence XML configuration file contains definitions of one or more related persistence items, such as all persistence items for a software component. The persistence XML file starts with a `<PERSISTENCE_CONFIG>` root tag to indicate that the file contains persistence item definitions. Within the `<PERSISTENCE_CONFIG>` construct there will be one or more `<resource>` tags. Each `<resource>` tag defines a single persistent item, such as a simple string, an integer, a file (for complex structures), etc. Within each `<resource>` construct there will be a set of tags that define attributes of the persistent item.

An extensive example of a persistence configuration XML file can be found in the Appendix.

Creating XML configuration file

Any number of file resources and key resources can be configured in the same Persistence XML file for a software component.

The XML file **must** contain the following persistent resource attributes (XML tags):

XML TAG DESCRIPTION	
TAG NAME	`<PERSISTENCE_CONFIG>`
DESCRIPTION	This XML root tag must encapsulate all defined persistent data resources.

XML TAG DESCRIPTION	
TAG NAME	`<resource>`
DESCRIPTION	Each persistent data resource's attribute tags must be encapsulated within this tag.

XML TAG DESCRIPTION	
TAG NAME	`<res_type>`
DESCRIPTION	Defines resource data type.
VALID VALUES	• `file` for file resources • `key-value` for key-value resources

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML TAG DESCRIPTION	
TAG NAME	`<name>`
DESCRIPTION	Defines the resource identifier for the data.
VALID VALUES	• Filename (e.g. `feature.conf`), NOT the full path, for file resources • key string name (e.g. `wifi_mac_address`) for key-value resources **NOTE:** This value is limited to only 50 characters.

XML TAG DESCRIPTION	
TAG NAME	`<application>`
DESCRIPTION	String name the software component will pass when calling `tpPCL_init(const gchar* application_name)` public API.
NOTE	*This is used for local data to grant access to it.* *For shared storage type, this value is used to differentiate between key-value resources with same name in the same database.* **NOTE:** This value is limited to only 50 characters.

XML TAG DESCRIPTION	
TAG NAME	`<storage_type>`
DESCRIPTION	Defines persistent data security access level.
VALID VALUES	• `public` for accessible by any process in the system • `group` for accessible by any process within the defined Linux group ID (either as primary or supplementary group) • `local` for accessible only by the owning process defined by application name
NOTE	1. *If `<storage_type>` is set to `public`, then `<user>` and `<group>` tags are ignored.* 2. *If `<storage_type>` is set to `group`, then `<user>` tag is ignored.*

XML TAG DESCRIPTION	
TAG NAME	`<permission>`
DESCRIPTION	Defines read/write permissions of persistent data.
VALID VALUES	• `rw` for data allowed for read and write operations • `ro` for data allowed for read-only operations

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML TAG DESCRIPTION	
TAG NAME	`<policy>`
DESCRIPTION	Defines the non-volatile commit policy of the data, including secured data (encrypted).
VALID VALUES	• `cached` to write changes to data in RAM and commit to non-volatile memory at a later time (recommended) • `write-through` to immediately write changes to data in non-volatile memory (only recommended for critical data) • `secure` to store the data encrypted on the file system and immediately write changes to data in non-volatile memory (encryption/decryption is managed internally within Persistence library so clients can use the same APIs, but actual data stored in non-volatile memory is secured)
CONSTRAINT	Please keep in mind that the size of each resource configured as *`secure`* is limited by the HAL implementation (see HAL_DECRYPT_DATA_SIZE_CHECK in Doxygen documentation).
NOTE	• *All data (key or file) that is configured as `secure` will take up more physical storage in non-volatile memory.* • `write-through` *and* `secure` *policies are atomic. Atomic write means either a write operation is successful or not done at all.*

XML TAG DESCRIPTION	
TAG NAME	`<user>`
DESCRIPTION	Defines the Linux user name ownership of the data for local storage. Protects read-write access permissions.
VALID VALUES	• Any valid Linux user name defined in the system **NOTE:** This value is limited to only 50 characters.

XML TAG DESCRIPTION	
TAG NAME	`<group>`
DESCRIPTION	Defines the Linux group name ownership of the data for group and local storage. Protects read-write access permissions.
VALID VALUES	• Any valid Linux group name defined in the system **NOTE:** This value is limited to only 50 characters.

XML TAG DESCRIPTION	
TAG NAME	`<data_type>`
DESCRIPTION	Defines the data type for how to read and write. For more details, see Data Types in XML Configuration Files.
VALID VALUES	• `binary` for key-value resources that are base64 string • `float` for key-value resources that are of float type • `string` for key-value resources that are NULL terminated string • `integer` for key-value resources that are of integer type (max 64-bit)
NOTE	*This tag is ignored for file resources.*

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML TAG DESCRIPTION	
TAG NAME	`<default>`
DESCRIPTION	Defines the default value to use for key-value resources.
VALID VALUES	• String, integer, float, or binary (Base64) value, based on `<data_type>` tag.
NOTE	• *For string `<data_type>`, it supports spaces; for empty strings, this tag requires at least one white space. Empty `<default>` tag is not supported.* • *For binary `<data_type>`, length of encoded data must be multiple of 4 characters.* • *This tag is ignored for file resources.*

XML TAG DESCRIPTION	
TAG NAME	`<default_size>`
DESCRIPTION	Defines the size of the default data in bytes.
VALID VALUES	• Actual file size for **file** resources • Length of the string including NULL terminator for key-value **string** resources • Number of bytes required for key-value **integer/float/binary** resources
NOTE	*For `secure` data, the size doesn't need to include the encryption data overhead.*

XML TAG DESCRIPTION	
TAG NAME	`<max_data_size>`
DESCRIPTION	Defines the maximum size in bytes that this persistent data can consume. Any write operation that causes the data to breach this limit will have undefined behavior.
NOTE	*For `secure` data, the size doesn't need to include the encryption data overhead.*

The XML file **can** contain the following optional persistent resource attributes (XML tags):

XML TAG DESCRIPTION	
TAG NAME	`<ldbid>`
DESCRIPTION	Optional tag. Defines if this resource is meant to be managed by a particular persistence plugin (e.g. data that needs to be stored in a specific non-volatile hardware). Custom plugins, such as plugins implemented by product team or 3rd party, define their own unique LDBID within this range.
VALID VALUES	• hex range "`0xF0000001`" – "`0xFFFFFFFD`"
NOTE	*`tpPCL_LogicalDatabaseId_t` enum in `persistence_client_library_types.h` public header defines which values are already taken by platform maintained plugins*
FDB - PLATFORM PLUGIN	〉 The hardware vendor provides a Factory Data Block plugin (reserved plugin LDBID - `0xFDB00FDB`) with read-only access. 〉 You CANNOT add Factory Data Block data using this tag in the same way you would create other plugin data. 〉 This FDB is flashed/installed specific to the hardware vendor solution.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML TAG DESCRIPTION	
TAG NAME	`<allow_override>`
DESCRIPTION	Optional tag. Defines if this resource is allowed to be overridden by product overrides. Used by platform Persistence configuration only. If not provided the Persistence resource installer will assume overriding is allowed. For more details, see Persistence Data Product Overrides.
VALID VALUES	• "0" to not allow product overrides • "1" to allow product overrides

XML TAG DESCRIPTION	
TAG NAME	`<rollback>`
DESCRIPTION	Optional tag. Defines if this resource requires a read-only default version to be able to roll back to if data is found to be corrupt to be explicitly requested to roll back to factory defaults.
VALID VALUES	• "0" if rollback is not needed • "1" to create default read-only rollback data to be managed by Persistence
NOTE	*Without configuring this `<rollback>` tag, resources will not have default data to fall back to if data becomes compromised or corrupt. This is the default behavior for resources without configuring this. Not including this will optimize storage space to not create redundant data (costing twice the storage space).*

XML TAG DESCRIPTION	
TAG NAME	`<delete>`
DESCRIPTION	Optional tag. ONLY to be used by Software Loading process for a field update to indicate the need to delete a resource (file or key-value) entirely from the system (including defaults) for use cases when software updates no longer require installed data.
VALID VALUES	• "0" reserved • "1" to delete a resource and its defaults (if any)
NOTE	*This tag is discarded if used in XML configuration files provided to the installer tool.*

XML TAG DESCRIPTION	
TAG NAME	`<force_update>`
DESCRIPTION	Optional tag. ONLY to be used by Software Loading process for a field update to indicate the need to completely update resource attributes for use cases when software updates require to change the default resource data.
VALID VALUES	• "0" compare the resource with existing one, if exists, update its attributes as needed, preserving its current data. • "1" update all resource attributes
NOTE	*When the tag is not present, default behavior is to preserve default resource data.* *This tag is discarded if used in XML configuration files provided to the installer tool.*

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML TAG DESCRIPTION	
TAG NAME	`<factory_only>`
DESCRIPTION	Optional tag. Protects persistence resources attributes that SHOULD NOT change after initial loading, meaning that resources will not be modified during an update process.
VALID VALUES	• "0" Resource can be updated. • "1" Resource attributes won't be modified during an update process.
NOTE	*When the tag is not present, default behavior is to allow resource attributes updates.* *When both <force_update> and <factory_only> tags are present, <factory_only> tag takes priority over <force_update>.*
XML TAG DESCRIPTION	
TAG NAME	`<enable_checksum>`
DESCRIPTION	Optional tag. Defines if checksum is enabled or disabled for file resources.
VALID VALUES	• "1" Checksum is enabled. • "0" Checksum is disabled.
NOTE	*For Key-Value pair checksum is always enabled. For file resources, when the tag is not present default behavior is checksum is disabled. However, checksum is always enabled for files which are created dynamically using tpPCL_fileOpen.*

2.2 Data Types in XML Configuration Files

For key-value resource types the `<data_type>` tag defines how the `<default>` tag value for the data is interpreted. All values in the XML configuration file are represented as text strings when the XML file is parsed. So, the `<data_type>` tag determines how to convert the text string value to the appropriate type for Persistence storage.

`integer` Data

Integer data type is interpreted as a hex (prefixed with "0x") or decimal string. Currently, the `<default_size>` and `<max_size>` support only values of 4 bytes (32-bit) or 8 bytes (64-bit).

For examples, see Appendix.

`string` Data

String data type is interpreted as a null terminated string exactly as defined in the XML configuration file. **The `<default_size>` and `<max_data_size>` values <u>must</u> include the NULL termination character in the size of the string data.**

For examples, see Appendix.

`float` Data

Floating point data type is interpreted as a float string. The `<default_size>` and `<max_size>` support only sizes of 4 bytes (32-bit) "float" and 8 bytes (64-bit) "double".

For examples, see Appendix.

`binary` Data

Binary data type is interpreted as a **base64** string (see https://en.wikipedia.org/wiki/Base64). This was chosen as a more efficient mechanism to configure large binary values without very large string representations in the

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

XML files. Many online tools are available to convert hex binary bytes to base64 string such as the following link that platform used for unit testing:

http://tomeko.net/online_tools/hex_to_base64.php?lang=en

The `<default_size>` value is the size of the actual binary data and **NOT** the base-64 encoded string.

For examples, see Appendix.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2.3 Build Process

Each software component will provide its own XML configuration files and file resources (if any) and install them under `$(SDK_ROOT_DIR)/opt/conti/share/persistence` directory.

To prepare a Persistence image to be installed on target, the files installed in this directory need to be archived. Here is an example of the command that can be used:

```
tar -czf tp-pers-install.tar.gz $(SDK_ROOT_DIR)/opt/conti/share/persistence/*
```

where `tp-pers-install.tar.gz` will be the name of the Persistence image.

2.3.1 HOWTO be a Client of Persistence Client Library

1) Use pkg-config in the client application Makefile to link to the PCL

```
LDFLAGS += `pkg-config --libs lcas-pcl`
```

2) Include Persistence Client Library header file in client application source code

```
#include "persistence_client_library.h"
```

3) **Security:** Usage of Persistence Client Library APIs require clients (user) to be a member of **"pers-group"** group.

4) Initialize communication with the Persistence Client Library in client application source code (**Note:** The client **must** first call `tpSYS_initIPC()`)

In case the user doesn't require any plugin to run, the second argument will be set to `NULL`:

```
tpPCL_init(application_name, NULL, TRUE);
```

In case the user requires a (list of) plugin(s) to initialize, see the following code snippet on how to build the `plugin_ldbids` argument:

```
const gchar *application_name = "my-app-name";
GArray *gaPluginIDs = NULL;
tpPCL_Error_t pcl_ret = E_PCL_ERROR_NONE;
tpPCL_LogicalDatabaseId_t uiLDbId = E_PCL_LDBID_PLUGIN_FDB;
g_array_append_val(gaPluginIDs, uiLDbId);

pcl_ret = tpPCL_init(application_name, gaPluginIDs, TRUE);
```

Products should ensure that PCL_PLUGIN_LIST platform key is overridden (if necessary) to match the PCL plugins (if any) installed on the target. If platform default plugins, such as "fdb", are removed from the package list then PCL_PLUGIN_LIST should also not include this plugin.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

See the following table for more details on `tpPCL_init()` API and its arguments.

tpPCL_init (const gchar *application_name, GArray * plugin_ldbids, gboolean receive_change_notification)	
application_name[in]	Unique client name to identify access to local application data
plugin_ldbids[in]	Array of ldbids for plugins required by the client. When this argument is NULL, PCL considers that the client does not use any PCL plugins for persisted data.
receive_change_notification[in]	Flag to specify if the client wants to receive and broadcast notifications. This flag **must** be `TRUE`! The option to set it to `FALSE` is used only for testing and performance purposes.

When finished using the persistence library, caller must make sure to call `tpPCL_deinit()` with no arguments to free any references and terminate process access to resources.

2.3.2 HOWTO be a Client of Persistence Admin Service

1) Use pkg-config in the client application Makefile to link to the PAS

```
LDFLAGS += `pkg-config --libs lcas-pas`
```

2) Include Persistence Admin Service header file in client application source code

```
#include "persistence_admin_library.h"
```

3) Wait for Persistence Admin Service to be ready at startup.
   a. If client application has a "hard" dependency on Persistence Admin Service to be running use systemd service "`After`" field

```
[Unit]
After=pas.service
```

   b. If client application has only a single thread (or portion of process) that is dependent on Persistence Admin Service use non-blocking TP service monitor APIs.

```
tpSVCMON_registerServiceStateChange(PAS_DBUS_SERVICE,       FALSE,
callback, &monitor_id);
```

4) **Security:** Usage of Persistence Admin Service APIs require clients (user) to be a member of **"pers-admin"** group.

5) Initialize communication with the Persistence Admin Service in client application source code (**Note:** The client **must** first call `tpSYS_initIPC()`)

```
tpPAS_Error_t pas_ret = E_PAS_ERROR_NONE;
pas_ret = tpPAS_init();
/* check return code */
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

When finished using the Persistence Admin Service, caller must make sure to call `tpPAS_deinit()` with no arguments to free any references and terminate process communication with it.

2.4 Tools/Equipment Needed

Persistence supports an installation tool to be used for factory installation of persistent data as well as for development. This tool accepts a file path to a Persistence image tarball as an argument.

The tool supports the following:

- parsing Persistence XML configuration files
- creating the Persistency file/folder structure
- setting the proper security permissions on the entire file/folder structure
- copying files
- creating databases
- overriding platform Persistence data with product Persistence data

The tool is called `pers-resource-installer` and it is accessible once the test partition is flashed.

2.5 Setup Instructions

Boot target in factory test mode

The target should be booted in factory test mode so that processes in the system do not try to read the persistent data while trying to install it.

Get persistence image onto target

The Persistence image needs to be pushed on target.

For the product that contain Android Debug Bridge (adb), the simplest way to handle this (for development and testing) is to use 'adb'.

Example (command to run on Windows host):

```
adb push <windows_path>\tp-pers-install.tar.gz /tmp
```

where `tp-pers-install.tar.gz` is the Persistence image created in the building process (See Build Process section for details).

Create persistence data on target

The persistence resource installer tool needs to be run to parse the persistence image tarball and create persistence resources as configured.

Command to run on target Linux console:

```
/tst/bin/pers-resource-installer /tmp/tp-pers-install.tar.gz
```

Reboot into normal mode

After the persistent data has been created on the target using the resource installer tool, running in normal mode will make use of Persistence data.

2.6 Setup Troubleshooting

The installer tool displays on console the result of installation.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

- In case no issue is encountered, `Successful installation` is displayed on console.
- In case something goes wrong, `Failed installation` is displayed on console, followed by an error code.

Troubleshooting in case of `pers-resource-installer` tool failures depends on the type of the error reported. Below, 2 common errors are addressed.

Invalid configuration files

One possible scenario is detecting issues with the file configuration format. The installation tool performs validation checks before installing the resources. In case the validation fails, the software component that owns the wrong XML configuration file needs to fix it. Persistence installation tool will display clear messages to point to the part of the file which is not correct. To get access to these detailed logs, use the `RI` grep filter on the log file.

The wrong XML configuration files are completely rejected. Keep in mind that the tool will still install the resources defined within correctly formatted XML files.

In this case, Persistence users should not rely on the installed data. Instead the errors should be fixed in the XML configuration files and the installation process should be repeated.

Encryption Issues

Another possible source of errors is the HAL setup. If the software components define secure data, the installation might fail in case `tpHAL_encryptData()` or `tpHAL_decryptData()` are failing, pointing to a setup issue. In this case, HAL setup needs to be performed before the tool is used to install the resources.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3 Features/Components

3.1 File Data

3.1.1 Overview

3.1.1.1 Platform Content Overview

Software components have the ability to configure files to be managed by Persistence. The actual location of the files in the file system is abstracted. This abstraction ensures that different files with the same file names are allowed if they are configured with a different `<storage_type>`.

3.1.1.2 Product Expectations

The software component needs to define the file resources using XML configuration files. It also needs to install these XML configuration files and the file resources under `$(SDK_ROOT_DIR)/opt/conti/share/persistence` at compile time.

The software component is expected to consistently use PCL file APIs to access, modify or delete the configured file resources, as opposed to using POSIX APIs.

3.1.1.3 Extending Platform Functionality

Not applicable.

3.1.2 Setup

Not applicable.

3.1.2.1 Tools/Equipment Needed

Not applicable.

3.1.2.2 Setup Instructions

Not applicable.

3.1.2.3 Setup Troubleshooting

Not applicable.

3.1.3 Usage

3.1.3.1 Basic Instructions

Once the file resources are installed on target by Persistence installer tool or by PAS, the software components have access to them through PCL file APIs.

3.1.3.2 Detailed Instructions

3.1.3.2.1 File Operations

This section lists all the file operations that PCL supports. More details can be seen following the format,

Doxygen Path

|-> Name of Data Type or API

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Where Doxygen Path is

```
Modules
      |-> Persistence
            |-> Persistence Client Library (PCL)
```

File Operations	
API Description	Name of APIs as Referenced in Doxygen Path
Enumeration of open file flags	`tpPCL_OpenFlags_t`
API to register for file change notifications.	`tpPCL_fileRegisterNotifyOnChange`
API to return a path where to dynamically store new files.	`tpPCL_getWriteablePath`
API to open a resource handle associated with a file.	`tpPCL_fileOpen`
API to get the size of the file in bytes. For `secure` files, this is the size of the decrypted file.	`tpPCL_fileGetSize`
API to read the data from a file.	`tpPCL_fileReadData`
API to reposition the file stream.	`tpPCL_fileSeek`
API to write data to a file.	`tpPCL_fileWriteData`
API to close the file handle.	`tpPCL_fileClose`
API to delete a file.	`tpPCL_fileRemove`
API to deregister for file change notifications.	`tpPCL_fileUnregisterNotifyOnChange`
API to map the file in the specified memory address range.	`tpPCL_fileMapData`
API to unmap a file from the specified address range.	`tpPCL_fileUnmapData`

3.1.3.2.2 File Resource Use Cases

HOWTO Dynamically Create a File Resource at Run-Time

Persistence supports dynamically creating file data at run-time for the use cases when we do not want or need to configure Persistence data at compile time, nor need the ability to rollback upon corruption detection.

In order to dynamically create file resources at run-time, a combination of `tpPCL_OpenFlags_t` is required with `E_PCL_OPEN_FLAG_CREATE` in the `flags` argument of `tpPCL_fileOpen()` API call. For example:

- `E_PCL_OPEN_FLAG_CREATE | E_PCL_OPEN_FLAG_RDONLY`, for read-only resources.
- `E_PCL_OPEN_FLAG_CREATE | E_PCL_OPEN_FLAG_WRONLY`, for write-only resources.
- `E_PCL_OPEN_FLAG_CREATE | E_PCL_OPEN_FLAG_RDWR`, for read / write permissions.

Files of type `secure` need to use the `E_PCL_OPEN_FLAG_RDWR` value in the `flags` argument.

HOWTO Get the Dir Path to the File Resource

In case the software component needs to store in Persistence a type of file that can't be handled using PCL file APIs (e.g. because it requires an open source package to handle it), it can use `tpPCL_getWriteablePath()` API to get a path, based on storage type, where the file is stored.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Example of a `.json` file in Persistence:

```
tpPCL_Error_t pcl_ret = E_PCL_ERROR_NONE;
gchar *dir_path = NULL;
gchar *file_path = NULL;
const gchar *filename = "my_conf.json";
gint size = 0;
/* Get the path prior to opening the file */
pcl_ret = tpPCL_getWriteablePath(E_PCL_LDBID_LOCAL, &dir_path, &size);

/* Build the file path using dir_path and filename */
file_path = g_strconcat (dir_path, filename);

if (pcl_ret == E_PCL_ERROR_NONE) {
      /* Provide the file_path as input to the opensource API to load the file */
      json_parser_load_from_file(parser, file_path, &error);
}

g_free (file_path);
```

3.1.3.3 Testing/Error Reporting

For an extensive list of error codes, see `tpPCL_Error_t` type definition. The following table explains some of these error codes in the context of file resource handling.

Error Code	Scenario
`E_PCL_ERROR_ERRNO`	Underlying APIs to `open`, `close`, `delete`, `map`, `unmap` or `move the cursor inside` a file fail. Checking if a missing file exists.
`E_PCL_ERROR_ACCESS_DENIED`	Trying to modify a read-only file or accessing a restricted file without the right permissions.
`E_PCL_ERROR_CORRUPT`	Corrupted file resource is being accessed.
`E_PCL_ERROR_NO_MEM`	Whenever Persistence fails to allocate memory dynamically.
`E_PCL_ERROR_RES_NOT_FOUND`	Trying to access a non-existent file.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.2 Key-Value Pair Data

3.2.1 Overview

3.2.1.1 Platform Content Overview

Software components have the ability to configure key-value pairs to be managed by Persistence. A key-value pair is a set of 2 linked data items: the **resource identifier** (the key) and the **data** associated with it (the value).

3.2.1.2 Product Expectations

1. The software component needs to define the key-value resources using XML configuration files. It also needs to install these XML configuration files under `$(SDK_ROOT_DIR)/opt/conti/share/persistence` at compile time.
2. The software component is expected to consistently use PCL key APIs to access, modify or delete the configured file resources, as opposed to using other APIs specific to the database.

3.2.1.3 Extending Platform Functionality

The users of Persistence can provide their own plugins to handle a custom data storage type, such as an external non-volatile storage hardware (think SD card).

The plugin needs to implement the Persistence plugin requirements so that PCL can forward to the plugin the requests coming from different software components. For details on how to implement a plugin, see TP_F100W_00006.

3.2.2 Setup

Not applicable.

3.2.2.1 Tools/Equipment Needed

Not applicable.

3.2.2.2 Setup Instructions

Not applicable.

3.2.2.3 Setup Troubleshooting

Not applicable.

3.2.3 Usage

3.2.3.1 Basic Instructions

Once the key resources are installed by Persistence, the users can access, modify and delete them through PCL key APIs.

3.2.3.2 Detailed Instructions

3.2.3.2.1 Key-Value Operations

This section lists all the key-value operations that PCL supports. More details can be seen following the format,

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Doxygen Path
|-> Name of Data Type or API

Where Doxygen Path is

```
Modules
    |-> Persistence
        |-> Persistence Client Library (PCL)
```

Key-Value Operations	
API Description	Name of APIs as Referenced in Doxygen Path
Function to dynamically create a key-value with requested attributes.	**`tpPCL_keyCreate`**
Function to delete a key and all of its data.	**`tpPCL_keyDelete`**
Function to get the size of data in bytes. For `secure` keys, this returns the size of the decrypted data.	**`tpPCL_keyGetSize`**
Function to read data for a key.	**`tpPCL_keyReadData`**
Function to register for key value change notifications.	**`tpPCL_keyRegisterNotifyOnChange`**
Function to deregister for key value change notifications.	**`tpPCL_keyUnregisterNotifyOnChange`**
Function to write data for a key.	**`tpPCL_keyWriteData`**

3.2.3.2.2 HOWTO Dynamically Create a Key-Value Resource at Run-Time

Persistence supports dynamically creating key-value resources at run-time for the use cases when we do not want or need to configure Persistence data at compile time, nor need the ability to rollback upon corruption detection.

The software component needs to use the PCL key API `tpPCL_keyCreate()` to create the key resource and provide all attributes to dynamically configure the data.

3.2.3.3 Testing/Error Reporting

For an extensive list of error codes, see `tpPCL_Error_t` type definition. The following table explains some of these error codes in the context of key-value resource handling.

Error Code	Scenario
`E_PCL_ERROR_INVALID_ARG`	Trying to read more information than the size of the stored data for the specified key. ***Note**: If the software component mentions a size equal or smaller than the stored data, Persistence will only return the number of bytes requested by the application and in case of mismatch, a log message is printed.*
`E_PCL_ERROR_READ_ONLY`	Trying to modify a read-only key resource.
`E_PCL_ERROR_ACCESS_DENIED`	Trying to modify a restricted key resource without having the right access permissions.
`E_PCL_ERROR_CORRUPT`	Trying to access a corrupted database (to read or write a key).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3 Data Access Permissions

3.3.1 Overview

3.3.1.1 Platform Content Overview

Persistence supports protecting access to installed data using Linux users and groups. Platform services and product applications can configure their own persistence data to be protected or shared in the following ways:

- `public` shared data (any process can read/write)
- `group` shared data (any process that is a member of the configured group can read/write)
- `local` private data (only processes running as the configured user and application name can read/write)

Data can also be configured and protected with read/write permissions or read-only permissions (for data that is never meant to be written to except by Software Loading or other privileged platform services).

Persistence users can configure the access permissions to the data using the `<permission>` tag and `<storage_type>` tag in the configuration files. For details, see the Configuration section.

For all product team's designated user(s) and / or group(s) added into the system requiring persistence resources access, user `pers` should be included as part of every newly created group.

3.3.1.2 Product Expectations

While configuring its resources, the software component needs to consider the following:

The software component must properly configure its resources using `<storage_type>` to limit the access to data based on its needs.

The software component must properly define the `<user>` and `<group>` of its configured resources.

The software component has the responsibility to properly set the `<application>` in the XML configuration file. In case multiple software components are defining resources with same key name (`<name>`) in a shared database (`public` or `group`), the `<application>` name set in the XML configuration file will be used to filter the right resource.

The software component has the option to further protect the read/write access to its resources by using `<permission>` tag.

3.3.1.3 Extending Platform Functionality

Not applicable.

3.3.2 Setup

Not applicable.

3.3.2.1 Tools/Equipment Needed

Not applicable.

3.3.2.2 Setup Instructions

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.2.3 Setup Troubleshooting

Not applicable.

3.3.3 Usage

3.3.3.1 Basic Instructions

The critical part of this component happens during design phase when the software component defines its resources and their access permissions. Once the software component defines this properly, Persistence will take care of restricting the access to the component's data.

The software components can control access to their data by using `<storage>`, `<permission>`, `<user>` and `<group>` tags in the XML configuration files.

3.3.3.2 Detailed Instructions

HOWTO Configure Public Data

Consider the following example of configuration for `public` key-value pair.

```
<resource>
   <res_type>key-value</res_type>
   <name>OTP_PERS_NETWORK_IP_ADDR</name>
   <application>publicapp</application>
   <storage_type>public</storage_type>
   <permission>rw</permission>
   <policy>cached</policy>
   <user>my_user</user>
   <group>my_group</group>
   <data_type>string</data_type>
   <default>192.168.0.1</default>
   <default_size>16</default_size>
   <max_data_size>16</max_data_size>
</resource>
```

By defining the resource as `public`, the software component can't limit the access to a specific user and group. Therefore, `<user>` and `<group>` values are ignored for public data. They are still deemed as **mandatory** by the XML schema and we recommend using a valid user and group from the user and group database, different than **`'root'`**.

On the other hand, the resource can be granted **read-only permissions** for everybody, thus blocking the users to modify it at run-time. This is an unusual scenario though; configuring the resources as read-write is the most common use case.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

HOWTO Configure Group Data

Consider the following example of configuration for `group` key-value pair.

```
<resource>
   <res_type>key-value</res_type>
   <name>FEATURE_VERSION</name>
   <application>my_app</application>
   <storage_type>group</storage_type>
   <permission>rw</permission>
   <policy>cached</policy>
   <user>my_user</user>
   <group>my_group</group>
   <data_type>string</data_type>
   <default>2.30.0.y</default>
   <default_size>9</default_size>
   <max_data_size>12</max_data_size>
</resource>
```

When defining a `group` shared resource, the access permission to the resource will be decided by `<group>` tag.

`<group>` **`'my_group'`** dictates who can access the data or not. Users that are part of this group as primary or supplementary group will be able to access and/or modify the resource.

`<user>` **`'my_user'`** doesn't impact the access to the resource. The `<user>` is **NOT** used in setting the access permissions for the `group` shared resources. The tag is mandatory in the XML schema though and we recommend using a user from the user database different than **`'root'`**.

`<permission>` tag can be used to block everybody to modify the resource at run-time. This is an unusual scenario though; the most common use case is granting read-write permissions to persisted resources.

HOWTO Configure Local Data

Consider the following example of configuration for `local` key-value pair.

```
<resource>
   <res_type>key-value</res_type>
   <name>PLUGIN_LIST</name>
   <application>my_app</application>
   <storage_type>local</storage_type>
   <permission>rw</permission>
   <policy>cached</policy>
   <user>my_user</user>
   <group>my_group</group>
   <data_type>string</data_type>
   <default>fdb</default>
   <default_size>4</default_size>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
    <max_data_size>32</max_data_size>
</resource>
```

When defining a `local` resource, the access permissions to the resource will be decided by the combination of the following tags: `<application>`, `<permission>`, `<user>` and `<group>`.

`<application>` **`'my_app'`** will be used to limit the access to only the software components that use the same `<application>` name to initialize the communication with Persistence (See `tpPCL_init()` API description).

`<user>` **`'my_user'`** and `<group>` **`'my_group'`** will be used as the owners of these resources.

`<permission>` can be used to block everybody (even the right owners) to modify the resource at run-time. This is an unusual scenario though; the most common use case is granting read-write permissions to persisted resources.

3.3.3.3 Testing/Error Reporting

For an extensive list of error codes, see `tpPCL_Error_t` type definition. The following table explains some of these error codes in the context of data access permissions.

Error Code	Scenario
`E_PCL_ERROR_READ_ONLY`	Trying to access a `read-only` resource.
`E_PCL_ERROR_ACCESS_DENIED`	Trying to access a `group` shared resource without the proper access permissions.
`E_PCL_ERROR_RES_NOT_FOUND`	• Trying to access another component's `local` data. • Trying to access a resource using an LDBID that doesn't correspond to the configured `<storage_type>`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.4 Data Change Notifications

3.4.1 Overview

3.4.1.1 Platform Content Overview

Persistence supports a notification mechanism to allow platform services and product applications to be able to register to be notified when data (file or key) has been changed. The notification does not include the data, so Persistence clients are expected to read the data if they want to know what has changed.

Once the notification mechanism is no longer needed, the software component can de-register so it doesn't receive the change notification anymore. This is unusual scenario; the most common use case is to register and stay registered until shutdown.

Persistence supports the following notifications:

Notification	Scenario
`E_PCL_NOTIFY_STATUS_CREATED`	Resource is created (When deferred created by PAS for SWL use case, or created by PCL client)
`E_PCL_NOTIFY_STATUS_DELETED`	Resource is deleted (When deleted by PCL client)
`E_PCL_NOTIFY_STATUS_MODIFIED`	Resource was modified
`E_PCL_NOTIFY_STATUS_CORRUPTED`	Resource is corrupted
`E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT`	• Resource was restored because of corruption. • Resource was restored on request.
`E_PCL_NOTIFY_STATUS_RESTORE_BACKUP`	Resource was restored to a backup.
`E_PCL_NOTIFY_STATUS_PREVIOUS_FILE_WRITE_ABORTED`	Previous resource write operation was aborted due to a crash/powercut.

This section addresses only `E_PCL_NOTIFY_STATUS_MODIFIED` notification. `E_PCL_NOTIFY_STATUS_CREATED` and `E_PCL_NOTIFY_STATUS_DELETED` are handled in a similar fashion too.

Registration for Persistence notification is done individually, per resource (file or key) and for all notifications. The user doesn't get to choose to receive only one notification or the other.

3.4.1.2 Product Expectations

The software component has to pass `receive_change_notification` argument as `TRUE` when calling `tpPCL_init()`. For details, see HOWTO be a Client of Persistence Client Library.

Once the notification is received, the software component needs to perform a read operation to get the changed value for the resource. The notification doesn't include this information.

3.4.1.3 Extending Platform Functionality

Not applicable.

3.4.2 Setup

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.4.2.1 Tools/Equipment Needed

Not applicable.

3.4.2.2 Setup Instructions

Not applicable.

3.4.2.3 Setup Troubleshooting

Not applicable.

3.4.3 Usage

3.4.3.1 Basic Instructions

Not applicable.

3.4.3.2 Detailed Instructions

3.4.3.2.1 Key-value Pair Change Notifications

1. Implement a callback for the Persistence notification for `key-value` resources to handle all types of notifications.

2. Register for the Persistence notification for key-value resource(s) using `tpPCL_keyRegisterNotifyOnChange()` API. **Please note that Persistence provides only one API to register for all type of notifications.**
3. The provided callback is called whenever an event for the specified `key` happens.

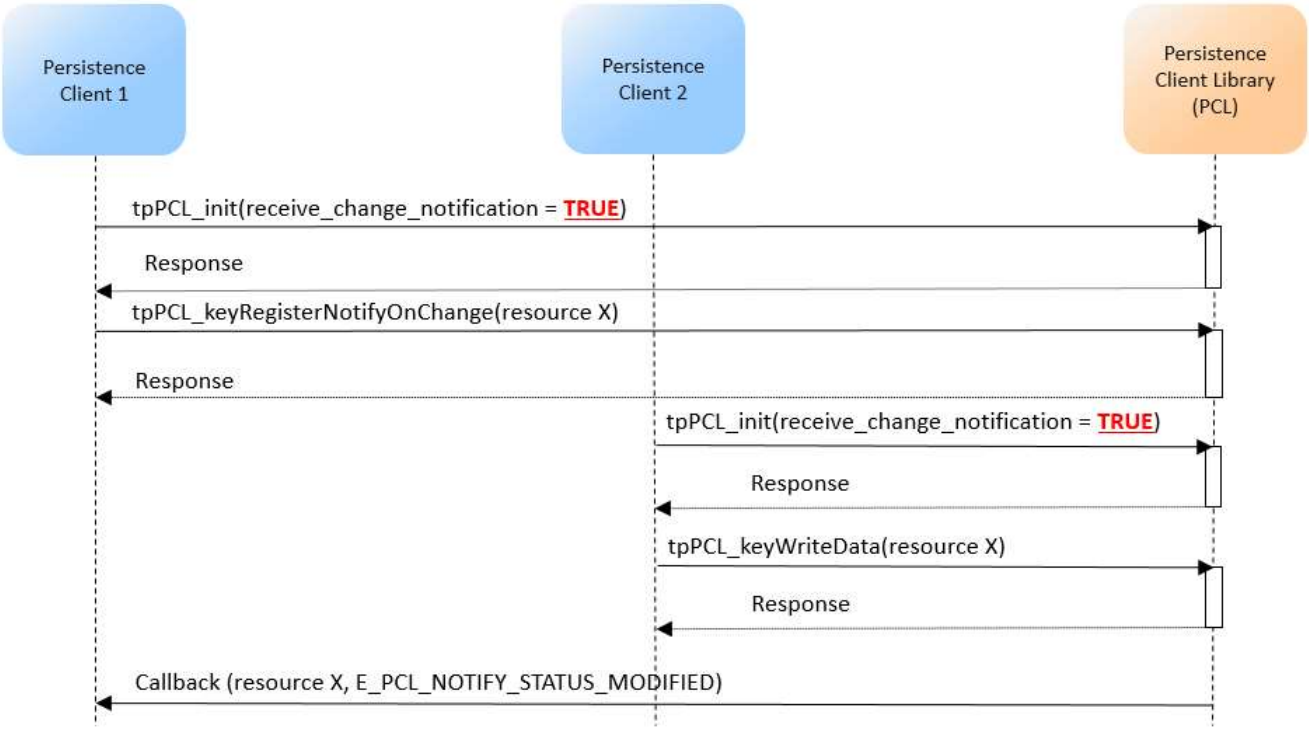


Figure 3 Data Change Notification Diagram

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

***Note**: If Persistence Client 2 would register for notifications for `resource X`, it will also get notified when the resource is modified, even though this client caused the change.*

3.4.3.2.2 File Change Notifications

1. Implement a callback for Persistence notification for `file` resources to handle all types of notifications.
2. Register the callback to receive Persistence file notifications using `tpPCL_fileRegisterNotifyOnChange()` API. **Please note that Persistence provides only one API to register for all type of notifications.**
3. Similar as for `key-value` pairs, the provided callback is called whenever an event for the specified `file` is triggered.

3.4.3.2.3 Resource Access Corrupted Notifications

1. The software component follows the step to register for Persistence notification. The callback needs to address the event `E_PCL_NOTIFY_STATUS_CORRUPTED`.
2. When the software component tries to access Persistence resources, Persistence will check for corruption. If the resource is corrupted, the API will return `E_PCL_ERROR_CORRUPT` error code. In addition to that, a corrupted event is broadcast to all registered clients.

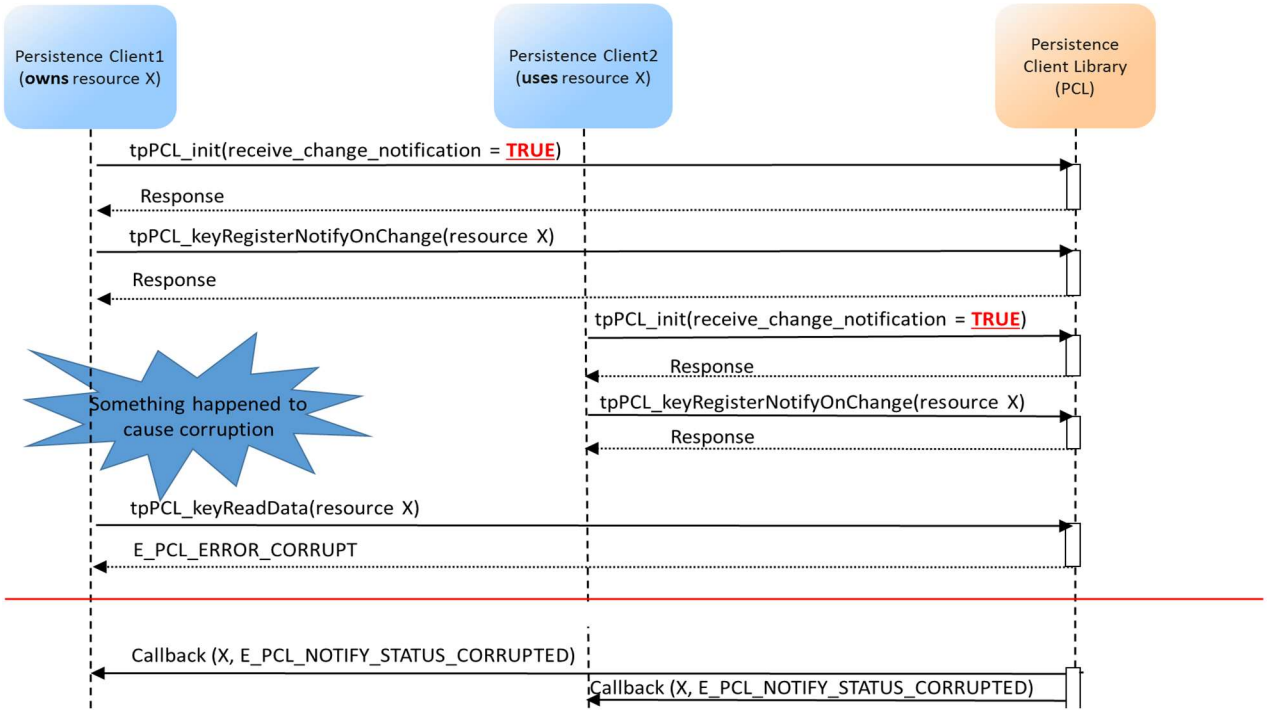


Figure 4 Key-Value Corruption Diagram

3.4.3.3 Testing/Error Reporting

If the software component doesn't enable notification mechanism when `tpPCL_init()` API is called, `E_PCL_ERROR_UNSUPPORTED` error code will be reported when `tpPCL_keyRegisterNotifyOnChange()` API is called.

If the software component doesn't enable notification mechanism when `tpPCL_init()` API is called, `E_PCL_ERROR_UNSUPPORTED` error code will be reported when `tpPCL_fileRegisterNotifyOnChange()` API is called.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.5 Data Corruption Detection and Rollback

3.5.1 Overview

3.5.1.1 Platform Content Overview

Persistence supports data (file or key) corruption detection and automatic recovery by falling back to configured default data.

Persistence supports a notification mechanism to allow platform services and product applications to be able to register to be notified when data (file or key) has been corrupted or restored.

If at run-time, Persistence identifies that the data can't be trusted, is corrupted, it will broadcast the `E_PCL_NOTIFY_STATUS_CORRUPTED` notification.

The `E_PCL_NOTIFY_STATUS_CORRUPTED` notification includes the path to the corrupted file or, respectively, the name of the key. A notification will be sent out for each monitored key from the corrupted database.

`<rollback>` tag is important for this use case. As described in 2.1 Configuration, the tag is used to decide which data needs backup. Only critical data should be configured using `<rollback>` tag. This will ensure an optimized storage space.

If the software component doesn't use the `<rollback>` tag or the data is dynamically created at run-time, Persistence won't be able to recover them in case of corruption.

Using this optional `<rollback>` tag with value "`0`" is the same as not using the tag at all.

If the data is configured with `<rollback>` or if the resource is part of a database where other resources are configured with `<rollback>`, Persistence will broadcast `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` notification once the resource (file or database) is restored. A recovered resource notification will be sent out for each monitored key from the recovered database.

An additional corruption detection mechanism is provided using checksums. The checksum feature is enabled by default for key-value resources, but it's optional for file resources. In order to enable file integrity checks using checksum, the software component needs to configure `<enable_checksum>` xml tag with value "`1`" at compile time. Using this optional `<enable_checksum>` tag with value "`0`" is the same as not using the tag at all.

The following limitations apply for file checksums:

1. **If checksum is enabled for a file, it can only be modified using tpPCL APIs**. Writing directly to the file, or to m-mapped memory (returned by tpPCL_fileMapData) will cause checksum validation failure in subsequent read/write using tpPCL APIs.
2. Checksum is always enabled for dynamically created file using tpPCL_fileOpen.
3. There is a non-linear performance penalty for enabling file checksums. File read/write performance penalty is less than 2% for files less than 2 MB, but the checksum computation time increases exponentially for larger files.

Persistence supports corruption detection mechanism from Persistence Admin Service. Also, it supports a mechanism to fetch list of corrupted persistence resource(s) at any time.

***Note**: This is more of a protection mechanism; this scenario wasn't encountered often in normal telematics testing.*

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.5.1.2 Product Expectations

In order to receive resource corruption notification client must call `tpPAS_registerNotifyOnCorrupt` API. To stop receiving resource corruption notification client must call `tpPAS_unRegisterNotifyOnCorrupt` API.

For this feature to work properly, the software component needs to configure its data properly using the `<rollback>` tag at compile time.

The software component needs to follow the steps to register for Persistence notifications (See 3.4 Data Change Notification). Persistence provides only one API to register for all types of notifications.

For `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` notification, Persistence clients are expected to read the data if they want to know what has changed.

In all cases where resources without `<rollback>` get corrupted, and Persistence broadcasts `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` notification, the software component that owns the resource is responsible to take action and recover the resource.

In case of a file resource that gets corrupted and Persistence broadcasts only `E_PCL_NOTIFY_STATUS_CORRUPTED` notification, the software component owner needs to take action to remove the resource and recreate it.

3.5.1.3 Extending Platform Functionality

Not applicable.

3.5.2 Setup

Not applicable.

3.5.2.1 Tools/Equipment Needed

Not applicable

3.5.2.2 Setup Instructions

Not applicable

3.5.2.3 Setup Troubleshooting

Not applicable

3.5.3 Usage

3.5.3.1 Basic Instructions

Persistence checks for corruption when a client tries to access the resource (file or key). Persistence is not handling keys separately, instead it checks if the whole database is corrupted, and if so, it tries to replace the whole database with the default one (if exists). Therefore, all keys that don't have default data will be lost.

3.5.3.2 Detailed Instructions

3.5.3.2.1 Key-Value Corrupted and Recovered Notifications

1. The software component follows the step to register for Persistence notification. See 3.4 Data Change Notifications section.
   The callback needs to address the 2 events, `E_PCL_NOTIFY_STATUS_CORRUPTED` and `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2. When the software component tries to access Persistence resources, Persistence will check for corruption. If the resource is corrupted, the API will return `E_PCL_ERROR_CORRUPT` error code. In addition to that, a corrupted event is broadcast to all registered clients.
3. Persistence will **automatically** trigger the recovery process at this point.
4. When the resource is recovered, Persistence will inform the software component and all registered clients asynchronously through an event that the data was restored to its default value. At this point, the provided callback will be called.
5. The action implemented in the provided callback for the `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` event will be executed.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

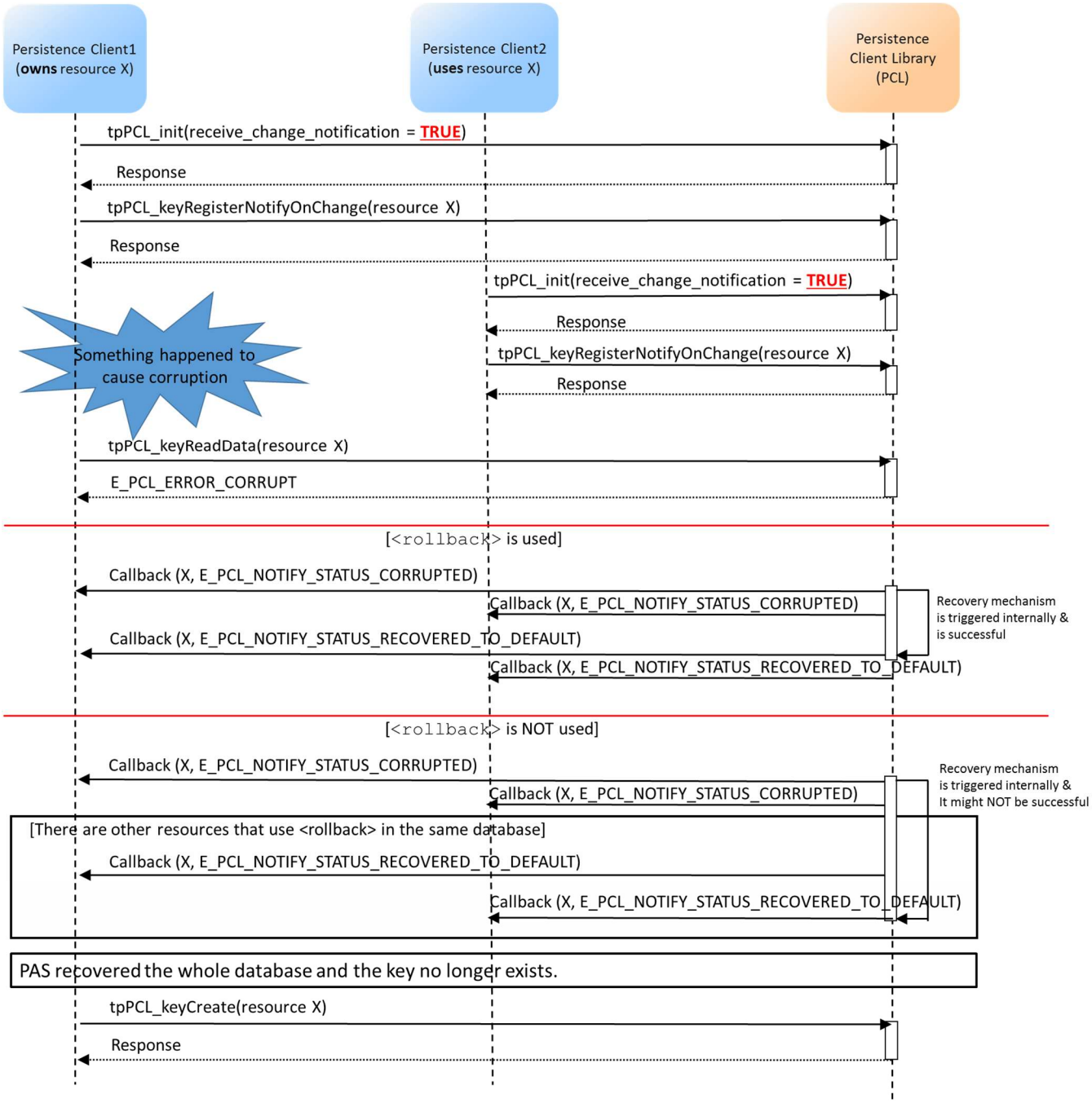


Figure 5 Corruption and Recovery Diagram

3.5.3.2.2 File Corrupted and Restored Notifications

1. The software component follows the step to register for Persistence file notification. See 3.4 Data Change Notifications section.

   The callback needs to address the 2 events, `E_PCL_NOTIFY_STATUS_CORRUPTED` and `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT`.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

2. When the software component tries to access Persistence `file` resources, Persistence will check for corruption. If the `file` resource is corrupted, the API will return `E_PCL_ERROR_CORRUPT` error code. In addition to that, a corrupted event is broadcast to all registered clients.
3. Persistence will **automatically** trigger the recovery process at this point.
4. When the `file` resource is recovered, Persistence will inform the software component and all registered clients asynchronously through an event that the data was restored to its default value. At this point, the provided callback will be called.
5. Similar to the `key-value` resources, the action implemented in the provided callback for the `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` event will be executed.

Also, in case the file doesn't use `<rollback>`, the owner is responsible for removing the file and recreating it.

Bellow is list of the operations that PAS supports regarding persistence resources corruption notification. More details can be seen following the format,

Doxygen Path
|-> Name of Data Type or API

Where Doxygen Path is

```
Modules
    |-> Persistence
        |-> Persistence Admin Service (PAS)
```

Persistence resources corruption notification	
API Description	Name of APIs as Referenced in Doxygen Path
API to register for resource corrupt notifications.	`tpPAS_registerNotifyOnCorrupt`
API to un-register for resource corrupt notifications.	`tpPAS_unRegisterNotifyOnCorrupt`
API to get array of corrupted resources since last power cycle.	`tpPAS_getCorruptedResources`

Note *Corrupted list is updated dynamically when resource is corrupted resource is accessed. List is maintained only for current power cycle.*

1. User may call `tpPAS_registerNotifyOnCorrupt` API to be notified any corruption..
2. User may call `tpPAS_getCorruptedResources` API to get any of corrupted resource already present.
3. User may call `tpPAS_unRegisterNotifyOnCorrupt` if corruption notification is no longer needed.

3.5.3.3 Testing/Error Reporting

If the software component doesn't enable notification mechanism when `tpPCL_init()` API is called, `E_PCL_ERROR_UNSUPPORTED` error code will be reported when `tpPCL_keyRegisterNotifyOnChange()` API is called.

If the software component doesn't enable notification mechanism when `tpPCL_init()` API is called, `E_PCL_ERROR_UNSUPPORTED` error code will be reported when `tpPCL_fileRegisterNotifyOnChange()` API is called.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.6 Persistence Data Product Overrides

3.6.1 Overview

3.6.1.1 Platform Content Overview

Persistence supports the ability of product applications to override platform persistence data (file and key) with their own values specific to the project (e.g. hardware specifics, calibration data, etc.). Platform provides default persistence data values such as configuration items, used for platform testing, that product teams need to override for their specific customer requirements.

Persistence also supports the ability of platform to configure Persistence data as "not allowed to be overridden" to prevent product applications from overriding data that they should not be allowed to.

This functionality is achieved thru the `<allow_override>` optional tag. In case this tag is missing from the XML configuration file, the default value is `'1'` (the software component is allowed to override the resource).

Note:

1. *These overrides happen during installation time and it is configured at compile time.*
2. ***This tag is meant for platform use only**. It has no impact if the non-platform software component uses it.*

3.6.1.2 Product Expectations

The software component can decide to modify platform resources based on customer requirements, by using one or more configuration files with the string `'override'` anywhere in the filename to specify the new value for those resources – example: `"persistence_feature_override.xml"`.

If the software component needs to override file resources, the override file resource needs to have same name as the original one, but with the added extension `'.pers-override'` - e.g. `platform.conf.pers-override`.

The platform software component must use `<allow_override>` as `"0"` if it must protect the data from product overrides during installation time.

3.6.1.3 Extending Platform Functionality

Not applicable.

3.6.2 Setup

Not applicable.

3.6.2.1 Tools/Equipment Needed

Not applicable.

3.6.2.2 Setup Instructions

Not applicable.

3.6.2.3 Setup Troubleshooting

Not applicable.

3.6.3 Usage

3.6.3.1 Basic Instructions

The software component needs to install the override XML configuration files and/or override file resources under the same directory - `$(SDK_ROOT_DIR)/opt/conti/share/persistence` (see Build Process).

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

These configuration files must adhere to the same XML configuration file standards as defined in section Configuration of this document, but the difference is that the resource installer tool handles persistent data creation/installation of these files last. Any number of key-value resources and file resources can be configured to be overridden in these XML configuration files.

For `key-value` resources, the resource configuration has to be identical in both XML configuration files, except the `<default>` tag, and if applicable, `<default_size>` and `<max_size>` tags. **All the other XML tags need to be left untouched!**

For `file` resources, `<name>`, `<default_size>` and `<max_size>` tags need to be updated in the override XML configuration file. **All the other XML tags need to be left untouched!**

3.6.3.2 Detailed Instructions

HOWTO Override Platform Data with Project Specific Data

Example of overriding a platform persistent key

Resource snippet of persistence XML configuration file:

```
<resource>
      <res_type>key-value</res_type>
      <name>HARDWARE_ID</name>
      <application>factory</application>
      <storage_type>public</storage_type>
      <permission>rw</permission>
      <policy>cached</policy>
      <user>pers</user>
      <group>pers-group</group>
      <data_type>string</data_type>
      <default>010</default>
      <default_size>4</default_size>
      <max_data_size>8</max_data_size>
      <allow_override>1</allow_override>
</resource>
```

Resource snippet of project override persistence XML configuration file (notice the value in `<default>` tag has changed):

```
<resource>
      <res_type>key-value</res_type>
      <name>HARDWARE_ID</name>
      <application>factory</application>
      <storage_type>public</storage_type>
      <permission>rw</permission>
      <policy>cached</policy>
      <user>pers</user>
      <group>pers-group</group>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
        <data_type>string</data_type>
        <default>000</default>
        <default_size>4</default_size>
        <max_data_size>8</max_data_size>
</resource>
```

Example of overriding a platform persistent file

Resource snippet of persistence XML configuration file:

```
<resource>
        <res_type>file</res_type>
        <name>platform_audio.cfg</name>
        <application>audio</application>
        <storage_type>local</storage_type>
        <permission>rw</permission>
        <policy>cached</policy>
        <user>audio</user>
        <group>audio</group>
        <data_type>binary</data_type>
        <default_size>4953</default_size>
        <max_data_size>5120</max_data_size>
</resource>
```

Resource snippet of project override persistence XML configuration file:

- notice the value in `<name>` tag has changed, the project team software component must maintain its own version of `platform_audio.cfg` configuration file as `platform_audio.cfg.pers-override` and install this file into `$(SDK_ROOT_DIR)/opt/conti/persistence/share`; this is to avoid filename collisions
- notice the values in `<default_size>` and `<max_data_size>` tags have changed to be accurate to the actual size of `platform_audio.cfg.pers-override` file in bytes and the maximum allowable size of this file

```
<resource>
        <res_type>file</res_type>
        <name>platform_audio.cfg.pers-override</name>
        <application>audio</application>
        <storage_type>local</storage_type>
        <permission>rw</permission>
        <policy>cached</policy>
        <user>audio</user>
        <group>audio</group>
        <data_type>binary</data_type>
        <default_size>5589</default_size>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
        <max_data_size>6000</max_data_size>
    </resource>
```

3.6.3.3 Testing/Error Reporting

While installing resources, if a software component tries to modify a resource which is protected by platform (`<allow_override>0</allow_override`), the installation won't exit with an error, it will only display an error message to inform the user that the override didn't work.

Example of the log message:

```
Platform doesn't allow the override of <name> resource
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.7 Persistence Backup and Restore

3.7.1 Overview

3.7.1.1 Platform Content Overview

Telematics Platform supports the ability to take a snapshot of persistent data during runtime as a backup copy. The backup copy will reside in the same database, for key-value resources, or the file system path, for file resources, until deleted. Persistence clients can restore data from any available snapshot at any time, which is notified to all clients (see Data Change Notifications section for more details on how to subscribe to these events notifications). Care should be given to ensure that the data is not being accessed (read / write) when attempting to restore a backup copy to prevent unintended behavior.

The number of backups for any persistence resource is only limited by available space on the device. Persistence backup design should consider additional storage space requirements for backup copies.

3.7.1.2 Product Expectations

Application development teams should keep track of the unique backup id by process for backed up resources. Different applications may back the same resource up, but different id will be generated.

3.7.1.3 Extending Platform Functionality

Not applicable.

3.7.2 Setup

Not applicable.

3.7.2.1 Tools/Equipment Needed

Not applicable.

3.7.2.2 Setup Instructions

Not applicable.

3.7.2.3 Setup Troubleshooting

Not applicable.

3.7.3 Usage

3.7.3.1 Basic Instructions

Not applicable.

3.7.3.2 Detailed Instructions

This section lists all the operations that PCL supports for backing up data and restoration to a backup value (for restoring the resource to the default read-only value see Restoring Defaults). More details can be seen following the format,

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Doxygen Path
|-> Name of Data Type or API

Where Doxygen Path is

```
Modules
    |-> Persistence
        |-> Persistence Client Library (PCL)
```

Resources backup and restore	
API Description	Name of APIs as Referenced in Doxygen Path
API to get a list of backups for a specific file resource.	**`tpPCL_fileBackupExist`**
API to take a backup of a file resource.	**`tpPCL_fileBackup`**
API to take a backup of a file resource with custom data.	**`tpPCL_fileBackupWithData`**
API to restore a file resource with backup data.	**`tpPCL_fileRestoreBackup`**
API to get a list of backups for a specific key resource.	**`tpPCL_keyBackupExist`**
API to take a backup of a key resource.	**`tpPCL_keyBackup`**
API to take a backup of a key resource with custom data.	**`tpPCL_keyBackupWithData`**
API to restore a key resource with backup data.	**`tpPCL_keyRestoreBackup`**

HOWTO take a backup snapshot of a resource

Whenever a resource is required to back its data up the next APIs can be called, depending on its type:

- For file resources:
  `tpPCL_fileBackup()` creates an exact copy of the resource in the same path, argument `backup_resource_id` returns the name of the backup.

- For key-value resources:
  `tpPCL_keyBackup()` creates an exact copy of the resource in the same database, argument `backup_resource_id` returns the name of the backup.

NOTE: `backup_resource_id` argument should be set to a pointer dereferencing `NULL` when a new backup snapshot is taken. If there's an existing backup id and is required to overwrite its value, this argument should be a pointer set to the backup id string. It's responsibility of the caller to free this pointer when is no longer required.

HOWTO take a backup snapshot of a resource with custom data

For software update scenarios, sometimes is required to save a backup of a resource with different data than the current value. These scenarios are uncommon, and its use shouldn't be abused by an application. The recommendation is to take backup snapshots of resources.

- For file resources:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

`tpPCL_fileBackupWithData()` creates a copy of the resource in the same path and writes the data of `buffer` instead of the actual contents of the file. Argument `backup_resource_id` returns the name of the backup.

- For key-value resources:
  `tpPCL_keyBackupWithData()` creates a copy of the resource in the same database and writes the data of buffer instead of the actual contents of the key. Argument `backup_resource_id` returns the name of the backup.

NOTE: `backup_resource_id` argument should be set to a pointer dereferencing `NULL` when a new backup snapshot is taken. If there's an existing backup id and is required to overwrite its value, this argument should be a pointer set to the backup id string. It's responsibility of the caller to free this pointer when is no longer required.

HOWTO get backup information of a resource

The following APIs can be used to obtain backup information for a resource depending on the resource type:

- For file resources:
  `tpPCL_fileBackupExist()` returns how many backups exist currently for file `resource_id`. `backup_count` argument holds an integer with the number of backups and `backup_resource_array` holds the list of backup ids.

- For key-value resources:
  `tpPCL_keyBackupExist()` returns how many backups exist currently for key `resource_id`. `backup_count` argument holds an integer with the number of backups and `backup_resource_array` holds the list of backup ids.

Both APIs will return arguments `backup_count` = `0` and `backup_resource_array` = `NULL` if there are currently no backups for the specific resource.

NOTE: This API is merely informative and is responsibility of the caller to free the backup resource array after its use.

HOWTO restore a resource to a specific backup value

To restore a resource to a specific backup value next APIs can be called, depending on its type. Persistence clients will receive a notification of type `E_PCL_NOTIFY_STATUS_RESTORE_BACKUP` whenever a resource is restored using any of these APIs.

- For file resources:
  `tpPCL_fileRestoreBackup()` restores the file to the backup contents of `backup_resource_id`.

- For key-value resources:
  `tpPCL_keyRestoreBackup()` restores the key to the backup contents of `backup_resource_id`.

NOTE: If `backup_resource_id` does not exist, then `E_PCL_ERROR_INVALID_ARG` is returned.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.7.3.3 Testing/Error Reporting

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.8 Restoring Defaults

3.8.1 Overview

3.8.1.1 Platform Content Overview

Persistent resources may have their own read-only copy known as “default value” that is intended to restore the resource in case of data corruption, but it is also possible to do so on-demand. For more information refer to rollback tag in Configuration section.

The Admin Service can roll the data back to its default for specific resources or all at once (for those the default value exists). The Client Library engine notifies to registered clients when this type of restoration occurs.

It’s possible to read this default value using PCL APIs for both file and key-value resources, but not modifying them in any way.

3.8.1.2 Product Expectations

The software component needs to use `<rollback>` tag when defining the resources that need a default value to roll back to.

The software component needs to follow preconditions for change notification to receive notification when the monitored resources are restored to default. See Data Change Notification.

The product should use these PAS APIs only for restoring defaults by privileged users (related to diagnostic, factory, or Software Loading). The use case when the restoration needs to occur should be based on customer requirements.

3.8.1.3 Extending Platform Functionality

Not applicable.

3.8.2 Setup

Not applicable.

3.8.2.1 Tools/Equipment Needed

Not applicable.

3.8.2.2 Setup Instructions

Not applicable.

3.8.2.3 Setup Troubleshooting

Not applicable.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.8.3 Usage

3.8.3.1 Basic Instructions

Not applicable.

3.8.3.2 Detailed Instructions

This section lists all the operations that the Admin Service supports restoring resources on demand. More details can be seen following the format,

Doxygen Path
|-> Name of Data Type or API

Where Doxygen Path is

```
Modules
      |-> Persistence
            |-> Persistence Admin Service (PAS)
```

Operations for restoring default values	
API Description	Name of APIs as Referenced in Doxygen Path
API to restore a resource to its default value.	**tpPAS_restoreDefault**
API to restore all resources to their default value.	**tpPAS_restoreAllDefaults**

For operations the Client Library supports for reading the default value contents of resources, the Doxygen path is:

```
Modules
      |-> Persistence
            |-> Persistence Client Library (PCL)
```

Operations for reading default values	
API Description	Name of APIs as Referenced in Doxygen Path
Definition for read-only file extension.	**PCL_DEFAULT_READONLY_RESOURCE_FILE_NEEDLE**
API to read the data size of the default value for key-value resources.	**tpPCL_keyGetDefaultSize**
API to read the data contents of the default value for key-value resources.	**tpPCL_keyReadDefaultData**

HOWTO Restore Individual Resources

Persistence Admin Service provides the `tpPAS_restoreDefault()` API for this operation. See Doxygen documentation for more details.

All Persistence clients registered to receive change notifications for this resource will receive a notification that the resource was rolled back to its default value - `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT`. Clients are responsible to read the new value for the restored resource.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

HOWTO Restore All Resources

Persistence Admin Service provides the `tpPAS_restoreAllDefaults()` API for this operation. See Doxygen documentation for more details.

Persistence will broadcast a `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` notification for each registered resource and to all registered clients. Clients are responsible to read the new value for the resources impacted by the restore operation.

NOTE: Dynamically created resources and resources without `<rollback>` configured are not affected by this operation. Nevertheless, Persistence will send a `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` notification for these resources.

HOWTO Read a default value for a key-value resource

For key-value default data, it's possible to know the exact size of it using `tpPCL_keyGetDefaultSize()` API. Knowing the size, actual default data can be retrieved using `tpPCL_keyReadDefaultData()` API. Use of these APIs is similar to `tpPCL_keyGetSize()` and `tpPCL_keyReadData()`.

NOTE: Proper API pairs should be used when reading modifiable data and default data. The size for default data may or not reflect the size of the current data and cause unexpected behavior.

HOWTO Read a default value for file resource

For files default data, regular APIs should be used by appending "`.default`" to the file resource name. `PCL_DEFAULT_READONLY_RESOURCE_FILE_NEEDLE` with this definition is available on the public header to be appended on a resource name for opening and reading default values.

3.8.3.3 Testing/Error Reporting

When using `tpPAS_restoreDefault()` API to request Persistence to restore the default value for a resource without a rollback value, `E_PCL_ERROR_RES_NOT_FOUND` error code is returned to indicate that the resource is missing rollback data.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.9 Persistence Data Field Update/Software Update

3.9.1 Overview

3.9.1.1 Platform Content Overview

Persistence supports adding, modifying and removing data during software update campaigns. Options are: direct manipulation (addition / removal) of data under Recovery mode, or through backup swapping under SSU mode. Both features accept XML diff configuration files to manipulate the data and may produce the same result with the proper XML structure. Also both features accept full XML configuration file and has the capability to compute the delta with existing resource configurations on the fly, and update the delta with preserving current value, if opted.

NOTE: Generating a diff configuration file that is consumed by PAS is currently NOT supported by Telematics Platform.

Special tags like `<delete>` is relevant when doing software updates and deprecated resources need to be removed; however this tag need to be used carefully to avoid losing critical data. The software gracefully ignores the tag, for a resource which is already deleted, i.e. not present.

The special tag `<force_update>` is used to configure the resource attribute update behavior as described in section 2.1 **NOTE: When tag is not present, default behavior is to preserve the default resource data**.

The special tag `<factory_only>` is used to protect persistence resource attributes that should not be changed after initial loading as described in section 2.1

Main features of direct manipulation recovery updates and backup swapping SSU updates are:

- Recovery mode updates accept a full persistence data image (multiple XMLs contained in persistence tarball).

  NOTE: Override feature (see Persistent Data Product Overrides section for more details) is not expected to work on software updates as it was designed only for resource installation.

- SSU updates accept a single XML file by design. All operations should be contained in this configuration file. If the application tries to include another XML, an error will be thrown.

- SSU updates preserves “old” and “new” data, making it possible to restore the previous software version if required.

- SSU updates can identify when a resource is modified in the new software version. Recovery updates would require removing the resource and adding it back with new data (adding a resource that is present will throw an error).

- SSU update also supports software update from any version to any other supported version, without the need of giving preconfigured “diff xml” as mentioned above. While the update is made, only the new resource configuration needed by the software-to-be-updated is used, and the resource delta is computed and updated on the fly.

- SSU update supports conditional preserving of resource current data, while the resource’s changeable attributes can be updated. For details on this behavior please refer the tables below.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

- SSU update supports adding resources for new users and groups (those are unknown to exiting software but available with the software update). For this special case, such resources are deferred installed after the new software is running – and the resource creation notification is broadcasted. Please see section 3.4.1.1 for the creation and deletion notification.

The behavior of software update preserving its current value is explained below:

- When any existing resource is compared against the given configuration by the update, any change in following attributes treats the resource as a "new" one to install: `<name>`, `<application>`, `<storage_type>`, `<user>`, `<group>`
- If `<data_type>` is changed (but the above-mentioned attributes match), the existing resource is completely updated as per the given configuration.
- When a resource is compared and updated, changes in following attributes are allowed: `<permission>`, `<policy>`, `<default>`, `<max_size>`. We will call this "delta" if such a change is present.
- If there is "delta", how the resource current value is worked out, is as below:

RW copy or RO (rollback) copy	**<force_update>**	**Resource data**
RW	1	Updated with the given data
RW	0	Preserved
RO (rollback)	0	Rollback copy updated with the given data
RO (rollback)	1	Rollback copy updated with the given data
RO (rollback) configured for existing data, but not configured for given data	1	Rollback copy is deleted
RO (rollback) configured for existing data, but not configured for given data	0	Rollback copy is preserved

- For "file" type resources the behavior is slightly different, since "delta" calculation attributes are not stored with the "file", the way it is stored in the database for "key-value" type resources. So, for file resource, if the resource exists, we assume "delta", and the behavior is set same as per the above table.
- Note that whenever "factory_only" tag is used data will always be preserved regardless of "force_update" tag value.

Using both approaches described before have limitations when including new users and groups and their associated private (local) and / or shared (group) data as part as the software update. This is currently not supported by the Telematics Platform.

NOTE: Both Recovery mode update and SSU mode update can't preserve current data when updating attributes only (e. g. changing default size, making data encrypted / unencrypted, changing ownership, etc.). In such scenarios, the value listed under the `default` tag overwrites the current data.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.9.1.2 Product Expectations

Product must create / maintain diff Persistence XML files for all required software updates between release versions.

Product must ensure that the proper diff Persistence XML file(s) is included in a Software Loading update package (see TP_F101U-00028).

3.9.1.3 Extending Platform Functionality

Not applicable.

3.9.2 Setup

Not applicable.

3.9.2.1 Tools/Equipment Needed

Not applicable.

3.9.2.2 Setup Instructions

Not applicable.

3.9.2.3 Setup Troubleshooting

Not applicable.

3.9.3 Usage

3.9.3.1 Basic Instructions

Not applicable.

3.9.3.2 Detailed Instructions

This section lists all the operations that PAS supports regarding modification of persistence resources on software updates. More details can be seen following the format,

Doxygen Path
|-> Name of Data Type or API

Where Doxygen Path is

```
Modules
      |-> Persistence
            |-> Persistence Admin Service (PAS)
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Data modification for software updates	
API Description	Name of APIs as Referenced in Doxygen Path
API to include / remove resources via XML configurations.	**tpPAS_resourceConfigAdd**
API to include / remove resources via XML configurations through non-blocking operation. The completion is conveyed later via the registered callback.	**tpPAS_resourceConfigAdd_async**
The completion callback to supply when using the async version of the API	**tpPAS_completionCB**
API to include / modify / remove resources as backups via XML configuration.	**tpPAS_resourceConfigAddAsBackup**
API to include / modify / remove resources as backups via XML configuration through non-blocking operation. The completion is conveyed later via the registered callback.	**tpPAS_resourceConfigAddAsBackup_a sync**
API to check if cached backups are active.	**tpPAS_isBackupActive**
API to swap existing data with cached backups.	**tpPAS_swapCachedBackups**
API to swap existing data with cached backups through non-blocking operation. The completion is conveyed later via the registered callback.	**tpPAS_swapCachedBackups_async**
API to free cached backups.	**tpPAS_freeCachedBackups**
API to free cached backups through non-blocking operation. The completion is conveyed later via the registered callback.	**tpPAS_freeCachedBackups_async**

HOWTO Add/Update/Delete resources in the field with Recovery updates

Platform provides an API to perform Persistence Update in the field – `tpPAS_resourceConfigAdd()`.

This is the API that expects the “diff” XML configuration file as input. This is the scenario when Persistence is looking for `<delete>` tags in the XML configuration files and removes existing data from persistent storage.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

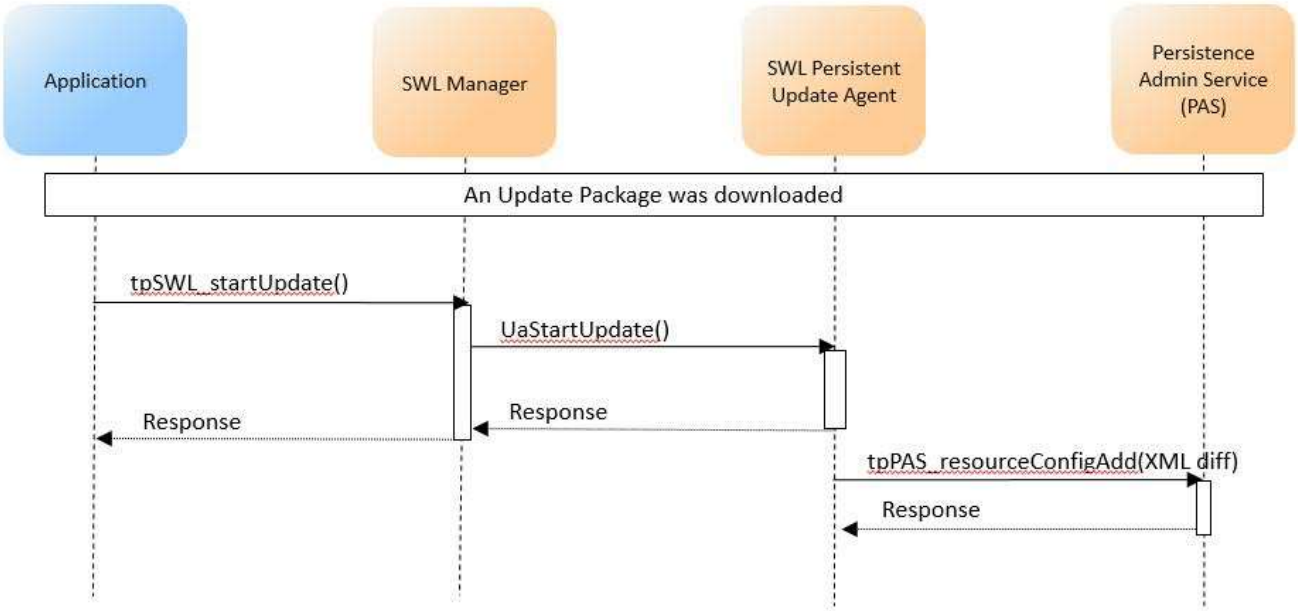


Figure 6 Persistence Update in the Field Diagram

HOWTO Add/Update/Delete resources in the field with SSU updates

There are software update scenarios where "old" data needs to be preserved because new firmware and associated persistent data are validated as part of the software update. If this validation fails, then the software update is rolled back, and old data is restored.

It's recommended to NOT use these APIs directly but to use Telematics Platform Persistence update agent instead.

`tpPAS_resourceConfigAddAsBackup()` API needs to be called first with an XML diff configuration as input. If the XML is formed correctly, the Admin Service will check the existence of each resource and save a backup of its current data (if it exists) or new data (if it not exists).

NOTE: Only one XML diff file can be used at a time for this kind of updates.

The Admin Service stores a list of cached backups with added, removed and modified data. The status of the cached backups list can be checked with `tpPAS_isBackupActive()` API, which shows the status if the cached list is active or not.

To toggle between active backups and current data, `tpPAS_swapCachedBackups()` API can be used. In case there's an error with the swapping, logic is in place to return to a known state of data, meaning there's no risk of having half swapped data. The same applies for unexpected power cuts, where Admin Service will request to complete the swap operation before continuing with any other action.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

When the proper data is selected, and the cached backups list is not required anymore, caller should use `tpPAS_freeCacheBackups()` API, which will remove the non-active data and any metadata stored in the file system.

Figure 7 shows a successful SSU software update scenario and the proper API calls to the Admin Service ending in accepting the new software and freeing the “old” data.

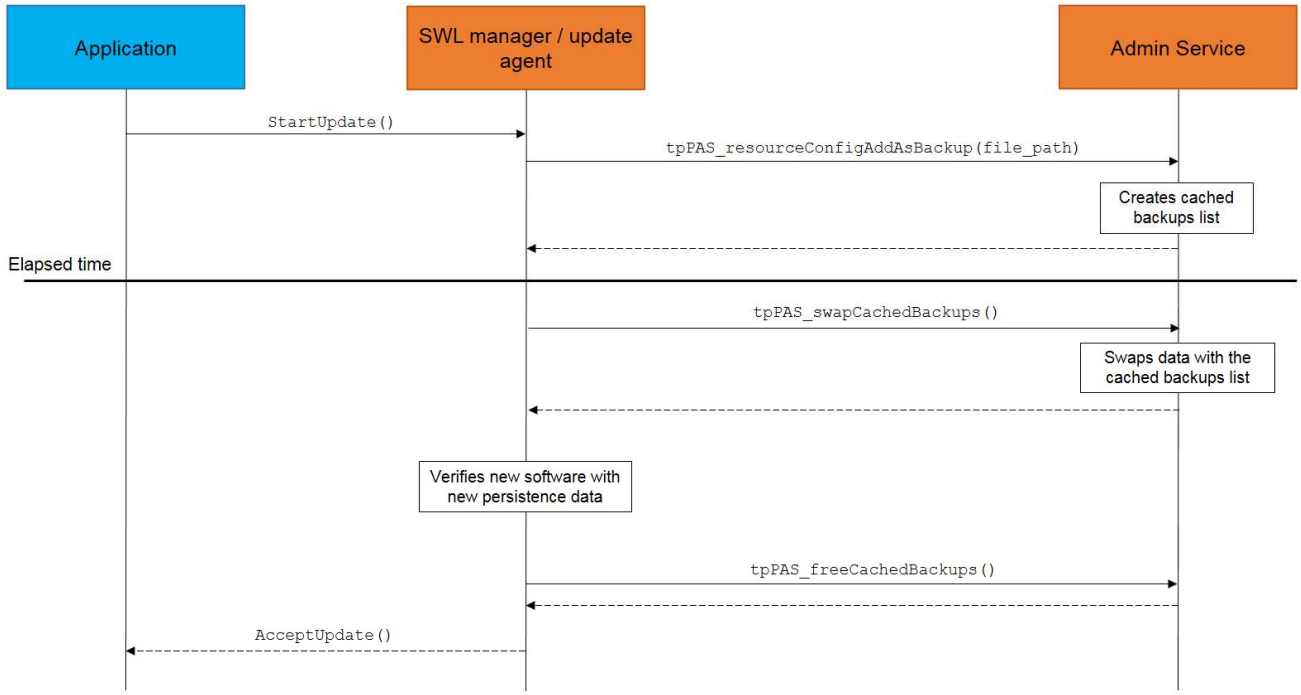


Figure 7. Scenario of a successful SSU software update

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Figure 8 shows an unsuccessful SSU software update scenario and the proper API calls to the Admin Service ending in rejecting the new software and rolling back to previous persistence data.

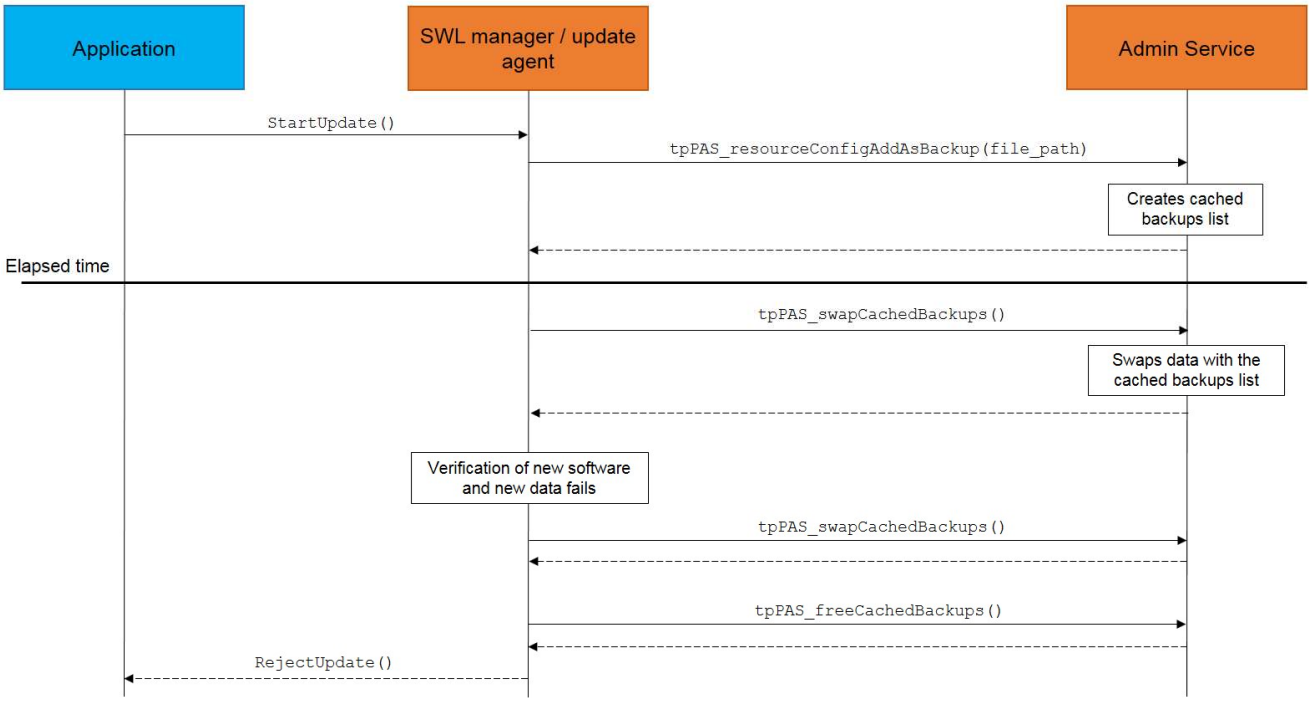


Figure 8. Scenario of an unsuccessful SSU software update

3.9.3.3 Testing/Error Reporting

If the provided XML configuration file doesn't have the right format, `E_PCL_ERROR_INVALID_XML` error is returned.

4 Known Issues

There is currently no support for `E_PCL_NOTIFY_STATUS_DELETED` notification. As a result, the following scenario will trigger an invalid notification.

- A client defines a key-value resource X without `<rollback>` tag.
- The client registers for notifications for resource X
- The entire key-value database where resource X resides gets corrupted and restored,

The client will be notified `E_PCL_NOTIFY_STATUS_RECOVERED_TO_DEFAULT` for resource X, but resource X won't exist in the newly recovered database because there was no rollback value. This means that the client is expected to re-create resource X with suitable value at the point when it receives this notification.

The goal is to broadcast a deleted event for this scenario. This part is not yet implemented.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

5 FAQs

5.1 Secure files

5.1.1 Opening a secure file for writing

Question:

Why the result for opening a secure file for writing is `E_PCL_ERROR_ERRNO`?

Answer:

When opening a secure file for updates with `tpPCL_fileOpen()` API, the `flags` argument required is `E_PCL_OPEN_FLAG_RDWR` from `tpPCL_OpenFlags_t`.

Using the flag `E_PCL_OPEN_FLAG_WRONLY` causes an `E_PCL_ERROR_ERRNO` error because PCL is trying to decrypt the data already stored in the file, and for that it needs read permissions.

6 Troubleshooting/Tips and Tricks

HOWTO Dynamically Create File Resources at Run-Time

Persistence test utility supports the option to exercise the PCL file API to create files dynamically.

```
pclfiletest: tpPCL_fileOpen ldbid resource_id handle policy flags
```

Example of creating a group file at run-time:

Request Persistence to create a new file by using `tpPCL_fileOpen()` API with `CREATE` flag set:

```
echo "tpPCL_fileOpen 1015 my_shared_file.txt 0 0 0x10" > /tmp/pcl_file_cmd
echo "tpPCL_fileClose handle" >> /tmp/pcl_file_cmd
PCL_TEST_APP_NAME=test pclfiletest FALSE -u power -g power -l NULL -f /tmp/pcl_file_cmd
```

HOWTO Dynamically Create Key Resources at Run-Time

Persistence test utility supports the option to exercise the PCL key API to create keys dynamically.

```
pclkeytest: tpPCL_keyCreate ldbid key_name rw_perm storage_policy data_type data_size max_size data
```

Example of creating a public key at run-time:

```
pclkeytest FALSE -l NULL tpPCL_keyCreate 0xfffffffe test-key 0 0 1 6 16 "hello"
```

HOWTO Delete a Key-Value Resource

Persistence test utility supports the option to exercise the API to delete an individual key-value resource.

```
pclkeytest: tpPCL_keyDelete ldbid resource_id
```

Example of deleting a public key:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
PCL_TEST_APP_NAME=my_app pclkeytest FALSE -l NULL tpPCL_keyDelete 0xfffffffe test-key
```

'my_app' needs to match the <application> name used in the configuration file and the value used in the tpPCL_init() call. In other words, the software component can delete **only** the data that it owns.

HOWTO Restore Individual Resources

Persistence test utility supports the option to exercise the API to restore an individual resource to its default value.

```
pastest: tpPAS_restoreDefault res_type ldbid resource_name app_name
```

Example of restoring a public key to its default value:

```
pastest tpPAS_restoreDefault 1 0xfffffffe OTP_PERS_NETWORK_IP_ADDR publicapp
```

7 References

7.1 Abbreviations/Acronyms/Definitions

PCL	Persistence Client Library
PAS	Persistence Admin Service

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F100W_00006_Writers_Guide_Persistence_Plugin
TP_F101U-00028_User_Guide_Software_Loading_Manager

7.3 Website links

Not applicable.

7.4 Appendix XML Configuration File Example

```
<?xml version="1.0" encoding="iso-8859-1" ?>
<!-- Copyright (c) 2016 Continental AG and subsidiaries                -->
<!-- The copyright notice above does not evidence any actual           -->
<!-- or intended publication of such source code.                      -->
<!-- Continental Confidential Proprietary Information.                 -->

<!-- TP Persistence XML resource configuration file -->
<!-- This file is just an example of different supported data types and different mandatory and optional tags -->

<PERSISTENCE_CONFIG>
      <!-- Example of a file resource configuration  -->
      <resource>
            <res_type>file</res_type>
            <name>my_package_config_file.conf</name>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
        <application>apm</application>
        <storage_type>local</storage_type>
        <permission>rw</permission>
        <policy>cached</policy>
        <user>apm</user>
        <group>apm</group>
        <default_size>4953</default_size>
        <max_data_size>5120</max_data_size>
        <allow_override>1</allow_override>
        <rollback >1</rollback>
</resource>

<!-- Example of an integer key resource configuration  -->

<resource>
        <res_type>key-value</res_type>
        <name>INT_KEY_EX</name>
        <application>swl-mgrd</application>
        <storage_type>local</storage_type>
        <permission>rw</permission>
        <policy>cached</policy>
        <user>swl</user>
        <group>swl</group>
        <data_type>integer</data_type>
        <default>600</default>
        <default_size>4</default_size>
        <max_data_size>4</max_data_size>
        <allow_override>1</allow_override>
</resource>

<resource>
        <res_type>key-value</res_type>
        <name>INT_MASK_KEY_EX</name>
        <application>pm</application>
        <storage_type>public</storage_type>
        <permission>rw</permission>
        <policy>cached</policy>
        <user>power</user>
        <group>power</group>
        <data_type>integer</data_type>
        <default>0x0800</default>
        <default_size>4</default_size>
        <max_data_size>4</max_data_size>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
</resource>

<!-- Example of a string key resource configuration  -->
<resource>
      <res_type>key-value</res_type>
      <name>STR_KEY_EX</name>
      <application>swl-mgrd</application>
      <storage_type>local</storage_type>
      <permission>rw</permission>
      <policy>cached</policy>
      <user>swl</user>
      <group>swl</group>
      <data_type>string</data_type>
      <default>One test string to rule them all</default>
      <default_size>33</default_size>
      <max_data_size>50</max_data_size>
</resource>

<!-- Example of a float key resource configuration  -->
<resource>
      <res_type>key-value</res_type>
      <name>FLT_KEY_EX</name>
      <application>swl-mgrd</application>
      <storage_type>local</storage_type>
      <permission>rw</permission>
      <policy>cached</policy>
      <user>swl</user>
      <group>swl</group>
      <data_type>float</data_type>
      <default>3.1415</default>
      <default_size>8</default_size>
      <max_data_size>8</max_data_size>
</resource>

<!-- Example of a binary key resource configuration. For details on how this
<default> data is converted to Base64, see HOWTO Define Binary Data section
below -->
<resource>
      <res_type>key-value</res_type>
      <name>BIN_KEY_EX</name>
      <application>swl-mgrd</application>
      <storage_type>local</storage_type>
      <permission>rw</permission>
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

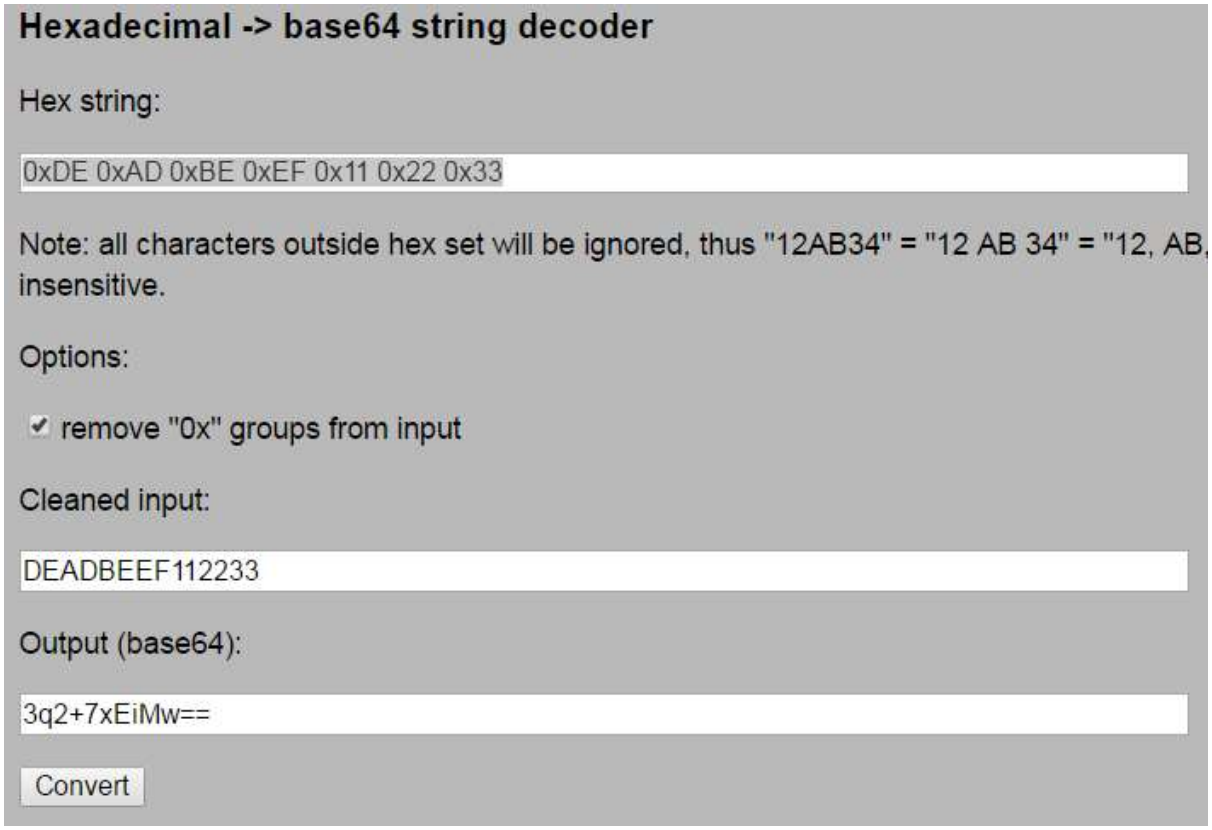
```
            <policy>cached</policy>
            <user>swl</user>
            <group>swl</group>
            <data_type>binary</data_type>
            <default>3q2+7xEiMw==</default> <!-- Length multiple of 4character -->
            <default_size>7</default_size>
            <max_data_size>7</max_data_size>
        </resource>

</PERSISTENCE_CONFIG>
```

HOWTO Define Binary Data

Example (store binary hex values 0xDE 0xAD 0xBE 0xEF 0x11 0x22 0x33):

(snippet from http://tomeko.net/online_tools/hex_to_base64.php?lang=en)



© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

8 Revision History

Version	Date	Description
01.00	Aug 15, 2019	Initial Release
02.00	Jul 11, 2020	Replace tpSYS service notify API with tpSVCMON API
03.00	Sep 11, 2020	General persistence updates for force_update and factory_only tags.
04.00	Jan 15, 2021	Updated persistence update for enable_checksum tag.
05.00	May 19, 2021	Resource corruption Notifications Updated enable checksum behavior for dynamically created files.
06.00	Jun 07, 2021	Atomic Write for Write-Through & Cached types.
		== ConMod-specific content from here ==
07.00	July 17, 2023	***Updated $3.5 with rollback and recovery functionality***

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted